

UNIT INFORMATION	
UNIT CODE	H7DT 04/VE/02
UNIT NAME	Web Application Development Project, Recorded Solution Demonstration, and Live Viva Voce Examination with Scenario-Based Coding Challenge
ASSESSMENT TYPE	PDF
PROJECT TITLE	

STUDENT INFORMATION	
STUDENT NAME	Kothalawala Liyanage Maleesha Pramud
STUDENT ID	2023/2024/CO/SE/I12/0837
BRANCH	Colombo
BATCH	2023/2024/CO/SE/Intake12 (ATHENA UNI)

Content

1. Introduction
2. Requirement Analysis
3. Website Planning
4. User Interface Design
5. HTML and CSS Development
6. PHP Development
7. Database Design and Operation
8. XML Implementation
9. Multimedia and File upload
10. Error Handling and validation
11. Screenshot of the Web Application
12. Conclusion
Appendices

1. INTRODUCTION

2nd Home is an online property rental platform. Property owners (hosts) publish houses and rooms with photos, prices and amenities, and travellers (guests) search the listings, check which dates are free, pay securely online and receive e-mail confirmation of their stay. Before this system, hosts had to rely on phone calls, social media messages and handwritten calendars to take reservations, which was slow and caused double bookings.

This document explains how the 2nd Home web application was planned, designed and built. The system lets guests browse and search properties, save favourites, add stays to a booking cart, check out in one payment and review the places they stayed in. Hosts get their own dashboard to manage listings and see the bookings they received. A separate administrator area manages every property, category, amenity, user and booking on the site.

The application was developed with PHP 8.3, MySQL 8, HTML5 and CSS3. The interface is built with the Bootstrap 5 framework and the Inter font, and uses JavaScript (AJAX with JSON), jQuery, Owl Carousel, DataTables and the TinyMCE rich-text editor. Property data is also published as an XML feed with a DTD. Payments use the PayHere payment gateway (sandbox mode) and e-mails are sent with PHPMailer. The project runs inside Docker (php:8.3-apache) and is served on port 8080.

1.1 Assumptions

The following ideas were considered in the building of the system:

- The system runs on an Apache, PHP 8.x and MySQL 8 server (provided here by Docker containers).
- There are three kinds of account: Admin, Producer (Host) and Customer (Guest). Anyone can register as a guest or a host; administrator accounts are created directly in the database.
- A property can only be booked by one guest for any given night, so overlapping date ranges are rejected.
- All prices are in Sri Lankan Rupees (LKR). A 10% service fee is added to the nightly price total.
- Guests identify themselves with a Sri Lankan NIC and a Sri Lankan mobile number (07X XXXXXXX) when booking.
- Payments are handled by the PayHere sandbox; no real money is transferred.
- Property and category photos are uploaded to the assets/images/ folder on the server.
- Notification e-mails (welcome, password reset, booking confirmation and cancellation) are sent through a Gmail SMTP account.
- A guest can only review a property that they have booked, and only once.

2. Requirements Analysis

2.1 Type Of Users

User Type	Who They Are	Access Level
Guest (Customer)	Travellers who register to search, book and review properties	Normal User (user_type_id = 3)
Host (Producer)	Property owners who list and manage their own properties	Host Access (user_type_id = 2)
Administrator	2nd Home staff who manage all data and users	Full Access (user_type_id = 1)
Visitor	Anyone who visits the site without logging in	Browse and search only

2.2 What Guests Need To Do

- Register an account (as a guest) and log in; reset a forgotten password by e-mail.
- Browse featured properties, newly added properties and property categories on the home page.
- Search by keyword and filter by category, maximum price and amenities.
- Open a property page to see its photo gallery, description, amenities, host and reviews.
- Check whether a property is available for chosen dates and number of guests.
- Save properties to a favourites list.
- Add several stays to a booking cart and pay for them together (multi-checkout), or book one stay directly.
- Pay online through PayHere and receive an e-mail confirmation.
- View, edit the details of, or cancel their own bookings.
- Write, edit and delete a review (rating 1–5 and comment) for a property they have booked.
- Update their profile details and change their password in Account Settings.

2.3 What Hosts Need To Do

- Register as a host (or upgrade from a guest account) and log in.
- View a dashboard with the number of own properties, bookings received and recent listings.
- Add a property with title, category, rich-text description, address, guests, bedrooms, beds, bathrooms, price, amenities and several photos.
- Edit or delete their properties, including removing and adding photos.
- Activate or deactivate a listing so that it appears or disappears on the site.
- View the bookings made on their own properties.

2.4 What Administrator Needs To Do

- Log in and open the admin dashboard to see summary numbers (properties, bookings, registered guests, hosts).
- View and manage every property on the site, and see every booking.
- Add, edit and delete property categories (with an image).
- Add, edit and delete amenities (with an icon).
- View all users, change a user's type, activate or deactivate an account, or delete a user.

2.5 Other Requirements

Security: All input must be validated on the server, special characters must be controlled, protected pages must check the logged-in user and role, and prices must be calculated on the server and never trusted from the browser.

Easy to use: The website must work on PCs, tablets and phones, with a clean layout and clear messages.

Reliable: All forms are validated twice (in the browser and on the server) and show clear error notifications. A property can never be double booked.

Easy to maintain: Reusable files are used for the database connection, validation rules, e-mail sending, header/navigation, sidebars and footer.

3. Website Planning

3.1 Purpose Of The System

Guests should be able to find a place to stay, see instantly whether it is free on their dates and pay for it online. Hosts can publish and manage their properties without technical help, and the administrator keeps the platform organised. The system uses a live MySQL database instead of manual files, so listings, prices and availability are always up to date.

3.2 Page List and Navigation

Page Name	File	Who can see it
Home page	index.php	Everyone
Search / Category results	search.php	Everyone
Property details	single-property.php	Everyone
Confirm and pay	booking.php	Everyone
Sign Up	signup.php	Everyone
Sign In	signin.php	Everyone
Forgot / Reset Password	forgot-password.php reset-password.php	Everyone
My Favorites	favorites.php	Everyone
Property XML feed	properties-feed.php	Everyone

Page Name	File	Who can see it
My Cart	cart.php	Logged-in users
My Bookings	my-bookings.php	Logged-in users
Account Settings	account-settings.php	Logged-in users
Host Dashboard	producer/dashboard.php	Host only
Properties (list, add, edit)	listing/list.php listing/add.php listing/edit.php	Host / Admin
Manage Bookings	listing/bookings.php	Host / Admin
Admin Dashboard	admin/dashboard.php	Admin only
Categories	admin/category/list.php add.php, edit.php	Admin only
Amenities	admin/amenity/list.php add.php, edit.php	Admin only
User Management	admin/user-management.php	Admin only

3.3 Benefits of the New System

- All data is stored in a database, so there is no need for handwritten calendars or spreadsheets.
- Guests can search, check availability and book at any time, from any device.
- Double bookings are prevented automatically because the server checks for overlapping dates.
- Hosts can publish and update their listings immediately without involving an administrator.
- Confirmation e-mails are sent automatically to guests and hosts.
- The system can be expanded easily when new properties, categories and amenities are added.

4. User Interface Design

4.1 Design choices

- The navigation bar is created once (components/NavigationBar.php) and included on every public page. Dashboards use a shared sidebar component.
- A calm slate blue (#445D7B) is used for the logo, primary buttons and table headers on a very light grey background.
- Card-style boxes with soft shadows are used for properties, price details and statistics.
- The layout uses the Bootstrap grid and adjusts automatically to tablets and smartphones.
- Forms have clear labels, large input fields and error messages shown as red alert boxes or pop-up (toast) notifications.

- The navigation menu changes depending on who is logged in (guest, host, admin or visitor).

4.2 Color Scheme

Color Name	Hex Code	Where It Is Used
Slate Blue (primary)	#445D7B	Logo, buttons, links, table headers, active states
Page Background	#F9FAFB	Background of every page
Surface White	#FFFFFF	Cards, navigation bar, forms, sidebar
Dark Text	#1F2937	Headings and body text
Gray Text	#6B7280	Small labels, dates, captions
Border Gray	#E5E7EB	Card borders and dividers

4.3 Page Layout Description

Home Page

A full-width hero slider with three background photos has a headline and a search bar (destination, check-in and check-out). Below it are category tiles, a three-column grid of top-rated property cards, an amenities strip, a "Just added" list, a trust section, guest testimonials, a host call-to-action banner and the footer.

Search Page

A filter sidebar (price range and amenities) sits on the left and property cards fill the right. On phones the filters move into a slide-in panel opened by a Filters button.

Property Page

A photo gallery grid at the top, then host details, description and amenities on the left. A sticky booking card on the right takes the dates and number of guests. Reviews are listed at the bottom.

Confirm and Pay Page

A two-column page: the left column shows the trip details and the guest details form; the right column shows the property and a price breakdown (price × nights, service fee and total).

Dashboards

A fixed sidebar on the left links to the other management pages. Statistic cards and data tables (with search, sorting and paging) appear on the right.

5. HTML and CSS Development

5.1 HTML5 Elements Used

The site uses HTML5 semantic elements to make the page structure clear and readable.

HTML Element	Where It Is Used
<header>	Site header with the logo, search box and user menu (NavigationBar.php)
<nav>	Main navigation, user drop-down menu and breadcrumbs
<main>	The main content area of each page
<section>	Groups related content such as the hero slider on the home page
<aside>	Dashboard sidebars and the search filter sidebar
<footer>	Four-column footer with links and newsletter box (Footer.php)
<form>	Sign in, sign up, search, filters, booking, listing and settings forms
<table>	Admin and host data tables with <thead> and <tbody>
<input> / <select> / <textarea>	Form fields including date, number, file, checkbox and password inputs
<img>	Property photos, category images and the logo

Example: the header and navigation from components/NavigationBar.php:

```
<header class="main-header">
  <div class="container">
    <nav class="main-nav">
      <a class="nav-brand" href="/index.php">
        <img src="/assets/images/logo/logo-only-text.png" alt="2nd Home" style="height: 40px;">
      </a>

      <form action="/search.php" method="GET" class="nav-search">
        <input type="text" name="q" placeholder="Search destinations, properties..." class="search-input">
        <button type="submit" class="search-btn">Search</button>
      </form>
      ...
    </nav>
  </div>
</header>
```

Code 5.1 – Semantic header, nav and form (components/NavigationBar.php)

5.2 External CSS Files

The styles are kept in external files inside assets/css/ and linked from components/head.php, so every page shares the same look. modern.css holds the global theme, navigation and cards; dashboard.css styles the admin and host dashboards; search.css, single-property.css and auth.css style the search page, property page and sign-in/sign-up pages. Bootstrap 5.3, Owl Carousel, DataTables and Bootstrap Icons are loaded as libraries. A few small inline styles are used for one-off layout tweaks (for example the hero slider on the home page).

```
<link rel="stylesheet" href="/assets/css/modern.css" />
<link rel="stylesheet" href="/assets/css/owl.carousel.min.css">
<link rel="stylesheet" href="/assets/css/owl.theme.default.min.css">
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css">
```

Code 5.2 – Stylesheets linked in components/head.php

5.3 CSS Features Used

CSS Variables (Custom Properties)

Variables hold the colours and common values at the top of modern.css, which makes it easy to change the theme in one place:

```
:root {
    /* "Nordic Soft" Color Palette */
    --primary-accent: #445d7b;
    --background-color: #F9FAFB;
    --surface-color: #FFFFFF;
    --primary-text-color: #1F2937;
    --secondary-text-color: #6B7280;
    --border-color: #E5E7EB;

    /* Font Weights */
    --font-regular: 400;
    --font-semibold: 600;
    --font-bold: 700;

    /* Other properties */
    --border-radius: 0.5rem;
}
```

Code 5.3 – CSS variables (assets/css/modern.css)

Types of Selectors Used

- Element selectors: body, h1–h6, a
- Class selectors: .card, .btn-primary, .main-header, .property-card-modern, .stat-card, .dashboard-sidebar
- Pseudo classes: :hover (cards, buttons, links, dropdown menu), :focus (search input), :first-child and :not(:first-child) (gallery)
- Descendant selectors: .hero-slider .item, .nav-dropdown:hover .dropdown-menu
- Pseudo elements: .hero-slider .item::after (dark overlay on the slider photos)

Layout and Responsive Design

Flexbox is used for the navigation bar and dashboard layout, and CSS Grid is used for the property photo gallery and amenities list. Media queries (min-width: 768px, min-width: 992px and max-width: 575.98px) change the layout on small screens: the gallery shows one photo, the booking card becomes static, and the search filters move into a slide-in panel.

```
.image-gallery {
    display: grid;
    grid-template-columns: 2fr 1fr 1fr;
    grid-template-rows: 1fr 1fr;
    gap: 0.5rem;
    height: 450px;
    border-radius: 12px;
    overflow: hidden;
}

/* on small screens */
    .image-gallery {
        grid-template-columns: 1fr;
        grid-template-rows: 1fr;
        height: 300px;
    }
    .gallery-item:not(:first-child) {
        display: none;
    }
```

Code 5.4 – CSS Grid gallery and its responsive rule (assets/css/single-property.css)

6. PHP Development

6.1 Variables

PHP variables store form data, query results and session data. Text input is trimmed and read with the null-coalescing operator so that missing fields never cause errors, and numeric values are validated before use:

```
// booking.php
$checkIn  = $_GET['checkIn'] ?? '';
$checkOut = $_GET['checkOut'] ?? '';
$guests   = $_GET['guests'] ?? '1';

// lib/add-listing-process.php
$title       = trim($_POST['title'] ?? '');
$category    = $_POST['category'] ?? '';
$description = trim($_POST['description'] ?? '');
$basePrice   = $_POST['basePrice'] ?? '';
$amenities   = isset($_POST['amenities']) && is_array($_POST['amenities']) ? $_POST['amenities'] : [];
```

Code 6.1 – Variables reading request data

6.2 Functions Used

Function	File	What It Does
session_start()	All pages and process files	Starts or resumes the user session
session_destroy()	logout.php	Ends the session when the user logs out
header()	Protected pages	Redirects users who are not allowed to see a page
trim()	All process files	Removes extra spaces from input
preg_match()	includes/validation.php	Checks names, mobile, email, NIC, numbers and text patterns
mb_strlen()	includes/validation.php	Checks text length limits
real_escape_string()	Database::escape()	Escapes special characters before values are used in SQL
htmlspecialchars()	NavigationBar.php, search.php, mailer.php	Stops script injection when printing user text
strip_tags()	Validation::sanitizeRichText()	Allows only simple formatting tags in descriptions
json_encode()	All lib/*-process.php files	Sends the result back to JavaScript as JSON
getimagesize() / is_uploaded_file()	Validation::validateImage()	Checks that an uploaded file is a real image
move_uploaded_file()	add-listing-process.php	Saves an uploaded photo to the server
uniqid()	Upload and order id code	Creates unique file names and order numbers
explode() / implode()	single-property.php, add-listing-process.php	Converts the image list between text and array
DOMDocument / createElement()	properties-feed.php	Builds and validates the XML feed

Function	File	What It Does
DateTime / diff()	Validation::getNights()	Validates dates and counts nights
date()	Pages with date inputs	Sets the earliest allowed date to today
md5()	checkout-process.php	Builds the PayHere payment hash
number_format() / round()	Booking and checkout code	Formats prices
unlink()	delete-listing-process.php	Deletes photo files when a property is removed
intval()	search.php, get-favorites-process.php	Forces IDs to integers

6.3 PHP Includes (Reusable Files)

Reusable files avoid copying the same code into every page. Every process file loads includes/connection.php, which in turn loads the validation and mailer classes; every page loads the shared components:

```
include './includes/connection.php';      // Database class + Validation + Mailer
include './components/head.php';          // <meta>, CSS links, page title
include './components/NavigationBar.php'; // header and navigation
include './components/Footer.php';        // footer
include './components/script.php';        // JavaScript libraries + toast container
```

Code 6.2 – Includes used by the pages

The dashboards include components/AdminSidebar.php or components/ProducerSidebar.php instead of the public navigation bar.

6.4 Forms and Data Transfer Between Pages

Forms send data with POST through small JavaScript helpers (AJAX). Values that identify a page, such as the property ID and the stay dates, travel between pages in the URL using GET:

```
// booking.php receives the property and stay details from the property page
if (isset($_GET['id']) && Validation::validateId($_GET['id'])) {
    $propertyId = $_GET['id'];
} else {
    header('location: /');
    exit();
}

$checkIn = $_GET['checkIn'] ?? '';
$checkOut = $_GET['checkOut'] ?? '';
$guests = $_GET['guests'] ?? '1';
```

Code 6.3 – Reading GET values (booking.php)

```
function PostRequest(path, formData, callback) {
    var req = new XMLHttpRequest();
    req.onreadystatechange = function () {
        if (req.readyState == 4) {
            if (req.status === 200) {
                var response = JSON.parse(req.responseText);
                if (response.status) {
                    callback(response);
                } else {
                    callback(null, response.message);
                }
            } else {
                callback(null, 'Request failed with status: ' + req.status);
```

```
            }
        }
    };
    req.open('POST', path, true);
    req.send(formData);
}
```

Code 6.4 – Sending form data with POST (assets/js/script.js)

6.5 Session Management

PHP sessions keep the user logged in while moving between pages. The e-mail address is stored in the session when the user signs in, and each protected page reads it to find the user and check their type:

```
// lib/signin-process.php – set at login
$_SESSION['email'] = $email;
$_SESSION['password'] = $password;

// producer/dashboard.php – check on a protected page
if (!isset($_SESSION['email'])) {
    header('Location: /signin.php');
    exit();
} else {
    $email = $_SESSION['email'];
    $userStmt = Database::search("SELECT * FROM `users` WHERE `email` = '$email'");
    $userData = $userStmt->fetch_assoc();
    if ($userData['user_type_id'] != 2) {
        header('Location: /index.php');
        exit();
    }
}

// logout.php
session_start();
session_destroy();
```

Code 6.5 – Sessions: login, page protection and logout

Inactive accounts (status_id ≠ 1) are refused at sign in. When the user ticks "Remember Me", the e-mail and password are also stored in cookies for 30 days so the sign-in form is pre-filled; logging out removes these cookies.

7. Database Design and Operation

7.1 Database Tables

The database is named online_store and has ten tables:

Table Name	What It Stores	Important Columns
users	Guest, host and admin accounts	id, fname, lname, mobile, email, password, vcode, user_type_id, status_id
user_types	Account roles	id, name (Admin, Producer, Customer)
status	Active / Inactive flag	id, name
categories	Property categories	id, name, image
amenities	Available amenities	id, name, icon_cls

Table Name	What It Stores	Important Columns
properties	Listed properties	id, title, categories_id, description, address, guests, bedrooms, beds, bathrooms, images, base_price, users_id, status_id
properties_has_amenities	Which amenities a property has	properties_id, amenities_id
bookings	Confirmed stays	id, checkIn, checkOut, first_name, last_name, nic, contact, guests, email, special_requests, properties_id, total_price, order_id
cart	Stays waiting to be checked out	id, users_id, properties_id, checkIn, checkOut, guests
reviews	Guest ratings and comments	id, properties_id, users_id, rating, comment, created_at

Relationships: properties belong to a category (categories_id) and a host (users_id); properties and amenities are linked many-to-many through properties_has_amenities; bookings, cart items and reviews each point to a property; reviews and cart items also point to a user and are removed automatically (ON DELETE CASCADE) when the property or user is deleted.

7.2 Connecting to the Database

A single Database class in includes/connection.php opens one shared MySQLi connection and offers helper methods for reading (search), writing (iud), escaping text and getting the last inserted id. (The host, user name and password are removed from the listing below.)

```
class Database {
    public static $connection;

    public static function setUpConnection() {
        if (!isset(Database::$connection)) {
            Database::$connection = new mysqli('host.docker.internal', 'root', '********', 'online_store', 3306);
        }
    }

    public static function search($query) {
        Database::setUpConnection();
        $rs = Database::$connection->query($query);
        return $rs;
    }

    public static function iud($query) {
        Database::setUpConnection();
        Database::$connection->query($query);
    }

    public static function escape($text) {
        Database::setUpConnection();
        return Database::$connection->real_escape_string($text);
    }

    public static function getInsertedId() {
        return Database::$connection->insert_id;
    }
}
```

```
require_once dirname(__FILE__) . '/validation.php';
require_once dirname(__FILE__) . '/mailer.php';
```

Code 7.1 – includes/connection.php

7.3 INSERT – Adding a Booking

After the input has been validated and the dates have been checked for overlap, the booking is saved. The total price is always calculated on the server:

```
// lib/booking-process.php
$calc = Validation::calculateBookingTotal($propertyId, $checkIn, $checkOut, $guests);
if (isset($calc['error'])) {
    echo json_encode(['status' => false, 'message' => $calc['error']]);
    exit();
}
$totalPrice = $calc['total'];

// Make sure the dates are still free
$overlap = Database::search("SELECT `id` FROM `bookings` WHERE `properties_id` = '$propertyId' AND `checkIn` <
'$checkOut' AND `checkOut` > '$checkIn'");
if ($overlap && $overlap->num_rows > 0) {
    echo json_encode(['status' => false, 'message' => 'This property is not available for the selected dates']);
    exit();
}

// Insert the booking into the database
Database::iud("INSERT INTO `bookings`
(`checkIn`,`checkOut`,`first_name`,`last_name`,`nic`,`contact`,`guests`,`email`,`special_requests`,`properties_id`,`total_pr
ice`,`order_id`) VALUES
('$checkIn','$checkOut','$firstName','$lastName','$nic','$contact','$guests','$email','$specialRequests','$propertyId','$totalPr
ice','$orderId')");
```

Code 7.2 – INSERT a booking (lib/booking-process.php)

7.4 SELECT – Featured Properties with Ratings

The home page joins properties with their reviews to show the average rating and the number of reviews of each active property:

```
// index.php
$propertiesStmt = Database::search("
    SELECT p.*, AVG(r.rating) AS avg_rating, COUNT(r.id) AS review_count
    FROM properties p
    LEFT JOIN reviews r ON p.id = r.properties_id
    WHERE p.status_id = 1
    GROUP BY p.id
    ORDER BY RAND()
    LIMIT 6
");
```

Code 7.3 – SELECT with JOIN, GROUP BY and LIMIT (index.php)

The search page builds its WHERE clause from the chosen filters (keyword, category, maximum price and amenities):

```
// search.php
if (!empty($searchTerm)) {
    $escapedSearch = Database::escape($searchTerm);
    $conditions[] = "(p.title LIKE '%$escapedSearch%' OR p.address LIKE '%$escapedSearch%')";
}
if ($categoryId) {
    $conditions[] = "p.categories_id = $categoryId";
}
$conditions[] = "p.status_id = 1";
...
if (!empty($conditions)) {
    $query .= " WHERE " . implode(" AND ", $conditions);
}
```

Code 7.4 – Dynamic search query (search.php)

7.5 UPDATE – Editing a Profile

```
// lib/update-profile-process.php
if (!isset($_SESSION['email'])) {
    echo json_encode(["status" => false, "message" => "Unauthorized access"]);
    exit();
}
...
Database::iud("UPDATE `users` SET `fname` = '$fname', `lname` = '$lname', `mobile` = '$mobile' WHERE `email` =
'$email'");

echo json_encode(["status" => true, "message" => "Profile updated successfully"]);
```

Code 7.5 – UPDATE the logged-in user's profile

7.6 DELETE – Cancelling a Booking

Before deleting, the script confirms that the booking belongs to the logged-in guest, then sends a cancellation e-mail:

```
// lib/delete-booking-process.php
$id = Database::escape($_POST['id']);

// Security check: Ensure this booking belongs to the logged-in user
$email = $_SESSION['email'];
$checkStmt = Database::search("SELECT b.*, p.`title` FROM `bookings` b JOIN `properties` p ON b.`properties  id` =
p.`id` WHERE b.`id` = '$id' AND b.`email` = '$email'");

if ($checkStmt->num_rows == 0) {
    echo json_encode(['status' => false, 'message' => 'Unauthorized access']);
    exit();
}
$booking = $checkStmt->fetch_assoc();

Database::iud("DELETE FROM `bookings` WHERE `id` = '$id'");

Mailer::sendBookingCancelled($booking['email'], $booking['first_name'], $booking['title'], $booking['checkIn'],
$booking['checkOut']);
```

Code 7.6 – DELETE a booking

7.7 Security

- Every value that is placed in an SQL statement is escaped with Database::escape() (mysqli real_escape_string), and IDs are checked against a whole-number pattern (Validation::validateId) or cast to integers.
- All input is validated on the server by the Validation class, even if the browser has already checked it.

- Prices are never taken from the browser. The server calculates the total from the property price, number of nights and the 10% service fee, and the PayHere payment hash is generated on the server.
- Text printed back on a page (for example the search term and e-mail content) is passed through htmlspecialchars(), and rich-text property descriptions are cleaned so that only simple formatting tags without attributes remain.
- Uploaded files are checked to be real images before they are saved, and are stored under a random unique file name.
- Dashboards check the session and the user type, and booking edit/cancel actions check that the booking belongs to the logged-in user.
- Double booking is prevented by an overlap check on every booking and checkout.

8. XML Implementation

The property data of 2nd Home is also published as an XML document (properties-feed.php). This can be used by mobile apps, partner websites and other systems that need to read the 2nd Home listings. The rules of the XML document are defined in a DTD file (xml/properties.dtd), and the system can check that the generated XML follows these rules.

8.1 XML Structure Files

The feed lists every active property with its category, description, address, capacity, price, rating, amenities and photos. The data comes live from the database, so the XML is always up to date. Opening properties-feed.php shows all active properties, and properties-feed.php?id=5 shows a single property. Example output:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE properties SYSTEM "/xml/properties.dtd">
<properties generated="2026-10-15" count="1">
  <property id="5">
    <title>Deluxe Room 1</title>
    <category>Rooms</category>
    <description>This is the description for deluxe room 1.</description>
    <address>40/2, Rathnapura Road, Munagama, Horana</address>
    <capacity guests="2" bedrooms="1" beds="1" bathrooms="1"/>
    <price>200.00</price>
    <rating reviews="0">0.0</rating>
    <amenities>
      <amenity>Wifi</amenity>
      <amenity>Kitchen</amenity>
    </amenities>
    <images>
      <image>/assets/images/properties/6a36b92a81a82.jpg</image>
    </images>
  </property>
</properties>
```

Code 8.1 – XML output of properties-feed.php?id=5 (shortened)

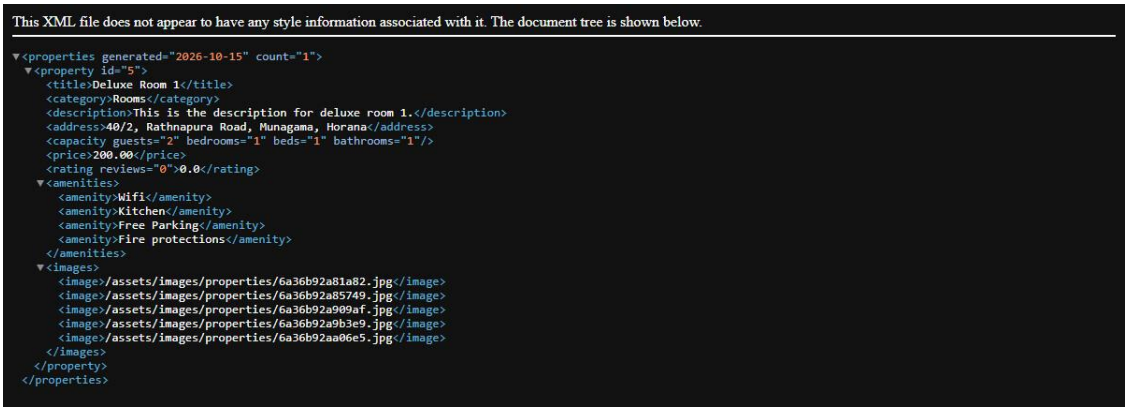


Figure 8.1 – XML feed shown in the browser (properties-feed.php?id=5)

8.2 Generating the XML with PHP

The feed is created with the PHP DOM extension (DOMDocument). The property rows are read from the database and each one is turned into XML elements and attributes. Text is added with createTextNode(), so special characters such as & and < are escaped automatically and the XML stays well-formed. The page is sent with the application/xml content type.

```
// properties-feed.php
$dom = new DOMDocument('1.0', 'UTF-8');
$dom->formatOutput = true;

$root = $dom->createElement('properties');
$root->setAttribute('generated', date('Y-m-d'));
$root->setAttribute('count', (string) $propertiesStmt->num_rows);
$dom->appendChild($root);

while ($row = $propertiesStmt->fetch_assoc()) {
    $property = $dom->createElement('property');
    $property->setAttribute('id', $row['id']);

    addText($dom, $property, 'title', $row['title']);
    addText($dom, $property, 'category', $row['category  name']);
    addText($dom, $property, 'address', $row['address']);

    $capacity = $dom->createElement('capacity');
    $capacity->setAttribute('guests', $row['guests']);
    $capacity->setAttribute('bedrooms', $row['bedrooms']);
    $property->appendChild($capacity);
    ...
    $root->appendChild($property);
}

header('Content-Type: application/xml; charset=UTF-8');
echo $xml;
```

Code 8.2 – Building the XML document (properties-feed.php)

8.3 Document Type Definition (DTD)

The DTD file lists which elements are allowed, in which order, and which attributes are required. The XML document points to it with the DOCTYPE line:

```
<!ELEMENT properties (property*)>
<!ATTLIST properties generated CDATA #REQUIRED
                     count     CDATA #REQUIRED>
```

```
<!ELEMENT property (title, category, description, address, capacity, price, rating, amenities, images)>
<!ATTLIST property id CDATA #REQUIRED>

<!ELEMENT title (#PCDATA)>
<!ELEMENT category (#PCDATA)>
<!ELEMENT description (#PCDATA)>
<!ELEMENT address (#PCDATA)>

<!ELEMENT capacity EMPTY>
<!ATTLIST capacity guests    CDATA #REQUIRED
                bedrooms  CDATA #REQUIRED
                beds      CDATA #REQUIRED
                bathrooms CDATA #REQUIRED>

<!ELEMENT price (#PCDATA)>
<!ATTLIST price currency CDATA "LKR"
               unit     CDATA "night">

<!ELEMENT rating (#PCDATA)>
<!ATTLIST rating reviews CDATA #REQUIRED>

<!ELEMENT amenities (amenity*)>
<!ELEMENT amenity (#PCDATA)>

<!ELEMENT images (image*)>
<!ELEMENT image (#PCDATA)>
```

Code 8.3 – xml/properties.dtd

8.4 Validating the XML

Adding ?validate=1 to the feed address (properties-feed.php?validate=1) makes the page load the generated XML together with the DTD and run DOMDocument::validate(). It reports whether the document follows the rules, or lists the errors if it does not. For example, a property that is missing its <category> element is rejected.

```
// properties-feed.php?validate=1
$checkDom = new DOMDocument();
$checkDom->loadXML($checkXml, LIBXML_DTDLOAD);

if ($checkDom->validate()) {
    echo "XML is valid against properties.dtd (" . $propertiesStmt->num_rows . " properties)";
} else {
    echo "XML is NOT valid:\n";
    foreach (libxml_get_errors() as $error) {
        echo '- ' . trim($error->message) . "\n";
    }
}
```

Code 8.4 – DTD validation



Figure 8.2 – Validation result (properties-feed.php?validate=1)

8.5 Why Use XML

- Other programs, such as mobile apps and partner websites, can read XML.
- It presents the listings in an orderly, readable form with a clear parent–child structure.

- The DTD guarantees that every mandatory field is present for each property record.
- It shows an alternative way of exchanging data outside the database and the website pages.

The AJAX requests between the pages and the server use the XMLHttpRequest object (see section 6.4) and exchange small JSON status messages such as {"status": true, "message": "..."}, because JSON is lighter for short replies. The site logo (assets/images/logo/logo_light.svg) is also an XML-based file (SVG). A Property XML feed link is available in the website footer.

9. Multimedia and File upload

9.1 Image Slider and Photo Gallery

The home page hero uses the Owl Carousel library to fade between three large background photos stored in assets/images/slider/. Every property page shows a photo gallery grid (one large photo and four smaller ones) with a "Show all photos" button that opens every uploaded image.

```
<section class="hero-section-modern">
   <div class="owl-carousel hero-slider">
      <div class="item" style="background-image: url('assets/images/slider/1.jpg');"></div>
      <div class="item" style="background-image: url('assets/images/slider/2.jpeg');"></div>
      ...
   </div>
</section>
```

Code 9.1 – Hero slider markup (index.php)

9.2 Property Photo Upload

Hosts can upload several photos when adding a property. The system checks every file before saving it:

Check	Rule
Upload error	The PHP upload error code must be UPLOAD_ERR_OK
File is a real upload	is_uploaded_file() must be true
File type	getimagesize() must succeed and the type must be JPEG, PNG, WebP or GIF
At least one image	A property cannot be saved without a photo

How the files are saved:

```
// lib/add-listing-process.php
foreach ($images['tmp_name'] as $key => $tmp_name) {
  if ($images['error'][$key] !== UPLOAD_ERR_OK || !Validation::validateImage($tmp_name)) {
    fail("Only valid image files (JPG, PNG, WEBP, GIF) are allowed");
  }
}

// Save images to the images folder and keep the file names
$imagePaths = [];
foreach ($images['tmp_name'] as $key => $tmp_name) {
  $name = uniqid() . ".jpg";
  $path = $RootFilePath . "/../assets/images/properties/" . $name;
```

```
  if (move_uploaded_file($tmp_name, $path)) {
    $imagePaths[] = $name;
  }
 }
$imagePathsString = implode(',', $imagePaths);
```

Code 9.2 – Validating and saving property photos

```
// includes/validation.php
public static function validateImage($tmpName) {
    if (!is_uploaded_file($tmpName)) return false;
    $info = @getimagesize($tmpName);
    return $info !== false && in_array($info[2], [IMAGETYPE_JPEG, IMAGETYPE_PNG, IMAGETYPE_WEBP,
IMAGETYPE_GIF]);
}
```

Code 9.3 – Image validation (includes/validation.php)

9.3 Editing and Deleting Photos

When a host edits a property, the existing photos are shown and can be removed (the kept file names are sent in keptImages) and new photos can be added. When a property is deleted, its photo files are removed from the server with unlink(). Category pictures are uploaded and deleted in the same way and stored in assets/images/categories/.

10. Error Handling and validation

10.1 Form Validation Rules

Field	Rule	Page
First / Last name	Letters and spaces only, maximum 20 characters	Sign up, Profile, Booking
Mobile	Sri Lankan mobile format, e.g. 0771234567 (07X + 7 digits)	Sign up, Profile, Booking
Email	Must be a valid e-mail format	Sign in, Sign up, Booking
Password	At least 5 characters to sign up / sign in; at least 8 characters when resetting or changing	Sign up, Reset, Change password
Account type	Must choose Guest or Host	Sign up
NIC	9 digits + V/X, or 12 digits	Booking
Check-in / Check-out	Valid dates, not in the past, check-out after check-in, maximum 365 nights	Property, Booking, Cart
Guests	Whole number from 1 up to the property's maximum guests	Property, Booking
Special requests	Optional, maximum 500 characters, no special symbols	Booking
Review	Rating 1–5; comment required, maximum 1000 characters; booked guests only, one review per property	Property page
Property title / address	Required; title maximum 100, address maximum 255 characters	Add / Edit property
Description	Required rich text, maximum 5000 characters, safe characters only	Add / Edit property

Field	Rule	Page
Guests, bedrooms, beds, bathrooms	Whole numbers from 1 to 100	Add / Edit property
Base price	Number greater than 0, up to 2 decimals	Add / Edit property
Photos	JPEG, PNG, WebP or GIF image files; at least one is required	Add / Edit property

10.2 How Errors Are Shown

Validation happens twice. First the browser checks the values with JavaScript regular expressions (RegexPatterns in script.js) and shows a message immediately. Then the server repeats the same checks with the Validation class, because browser checks can be bypassed. Server messages come back as JSON and are shown to the user either in a red alert box above the form (sign in and sign up) or in a pop-up toast notification (booking and management pages).

```
// includes/validation.php
const MOBILE_PATTERN = '/^07[0,1,2,4,5,6,7,8]{1}[0-9]{7}$/';
const NIC_PATTERN    = '/^(?:[0-9]{9}[vVxX]|[0-9]{12})$/';

public static function validateMobile($mobile) {
    return (bool) preg_match(self::MOBILE_PATTERN, $mobile);
}

// lib/signup-process.php
} else if (empty($fname)) {
    echo ("First name is required");
} else if (!Validation::validateName($fname)) {
    echo ("First name must contain only letters");
} else if (!Validation::validateMobile($mobile)) {
    echo ("Mobile is not valid");
} else if (!Validation::validateEmail($email)) {
    echo ("Email is not valid");
```

Code 10.1 – Server-side validation

```
// assets/js/script.js – client-side check before sending the booking
if (!nic) {
    showToast("NIC is required.", 'warning');
    return;
} else if (!RegexPatterns.nic.test(nic)) {
    showToast("NIC is not valid.", 'warning');
    return;
}
```

Code 10.2 – Client-side validation with toast notification

10.3 File Upload Errors

```
if ($images['error'][$key] !== UPLOAD_ERR_OK || !Validation::validateImage($tmp_name)) {
    fail("Only valid image files (JPG, PNG, WEBP, GIF) are allowed");
}
...
if (empty($imagePaths)) {
  fail("At least one image is required");
}
```

Code 10.3 – Upload error handling (lib/add-listing-process.php)

10.4 Business Rule Errors

```
// Dates already taken
if ($overlap && $overlap->num_rows > 0) {
    echo json_encode(['status' => false, 'message' => 'This property is not available for the selected dates']);
    exit();
}

// Only guests who booked may review
if ($bookingStmt->num_rows == 0) {
    echo json_encode(['status' => false, 'message' => 'You can only review properties you have booked.']);
    exit();
}
```

Code 10.4 – Availability and review rules

10.5 Page Access Control

If the user is not logged in, protected pages redirect to the sign-in page. If a logged-in user does not have the right role, they are sent back to the home page:

```
// Logged-in page protection (cart.php, my-bookings.php, account-settings.php)
if (!isset($_SESSION['email'])) {
    header('Location: /signin.php');
    exit();
}

// Admin page protection (admin/dashboard.php)
if ($userData['user_type_id'] != 1) {
    header('Location: /index.php');
    exit();
}
```

Code 10.5 – Page access control

11. Screenshots of the Web Application

All screenshots were taken from the running application (http://localhost:8080).

11.1 Home Page

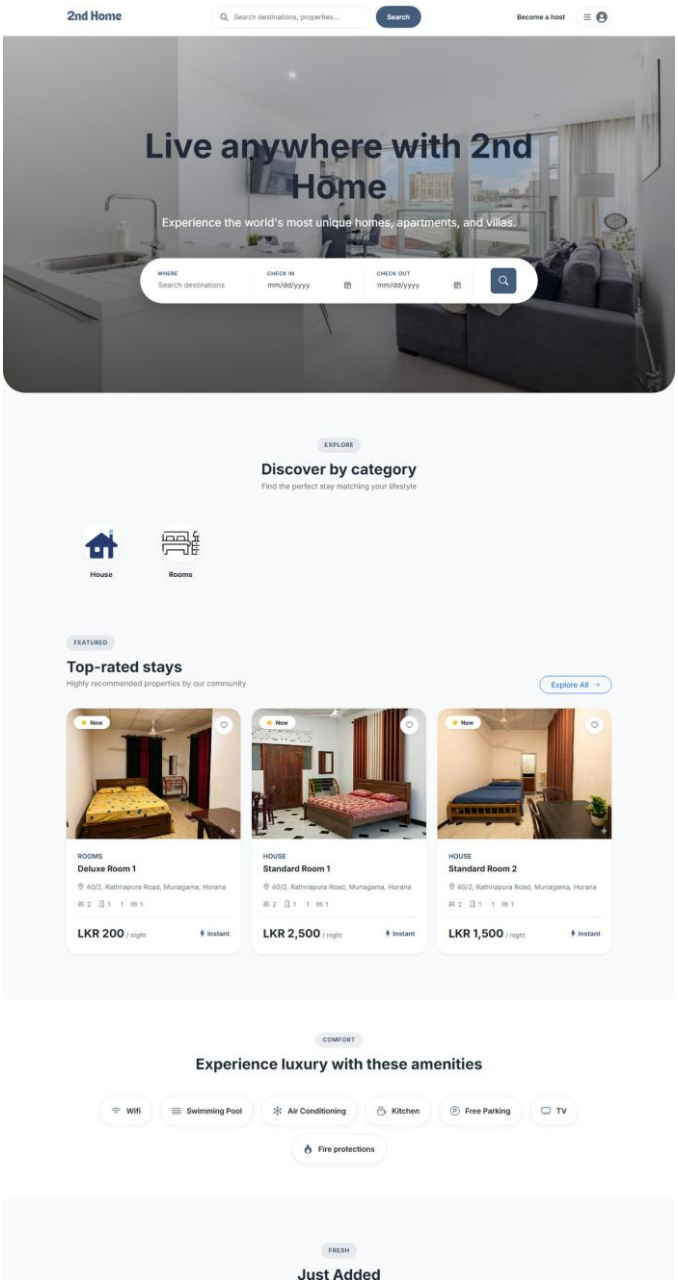


Figure 11.1 – Home page (hero slider, categories, featured and newest properties)

11.2 Search Results Page

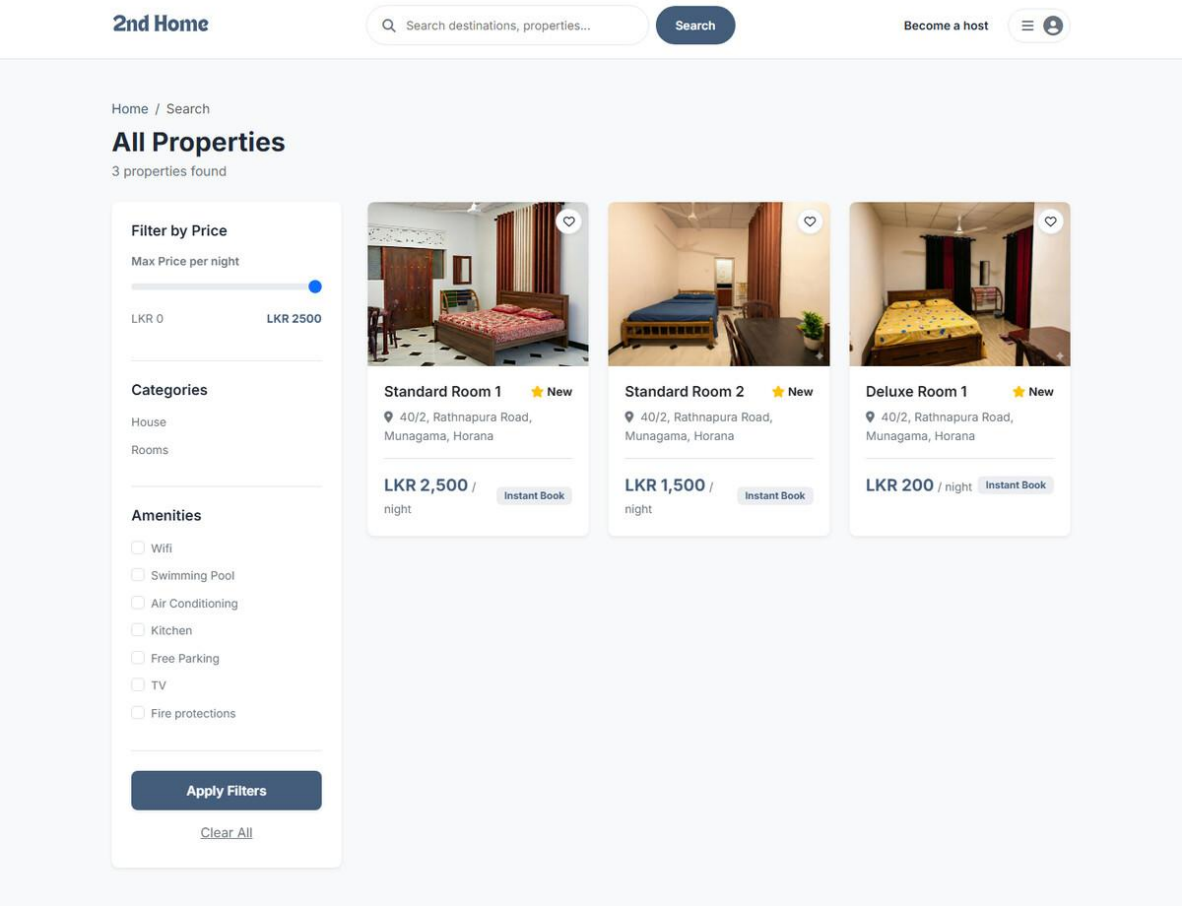


Figure 11.2 – Search page with filters and property cards

11.3 Property Details Page

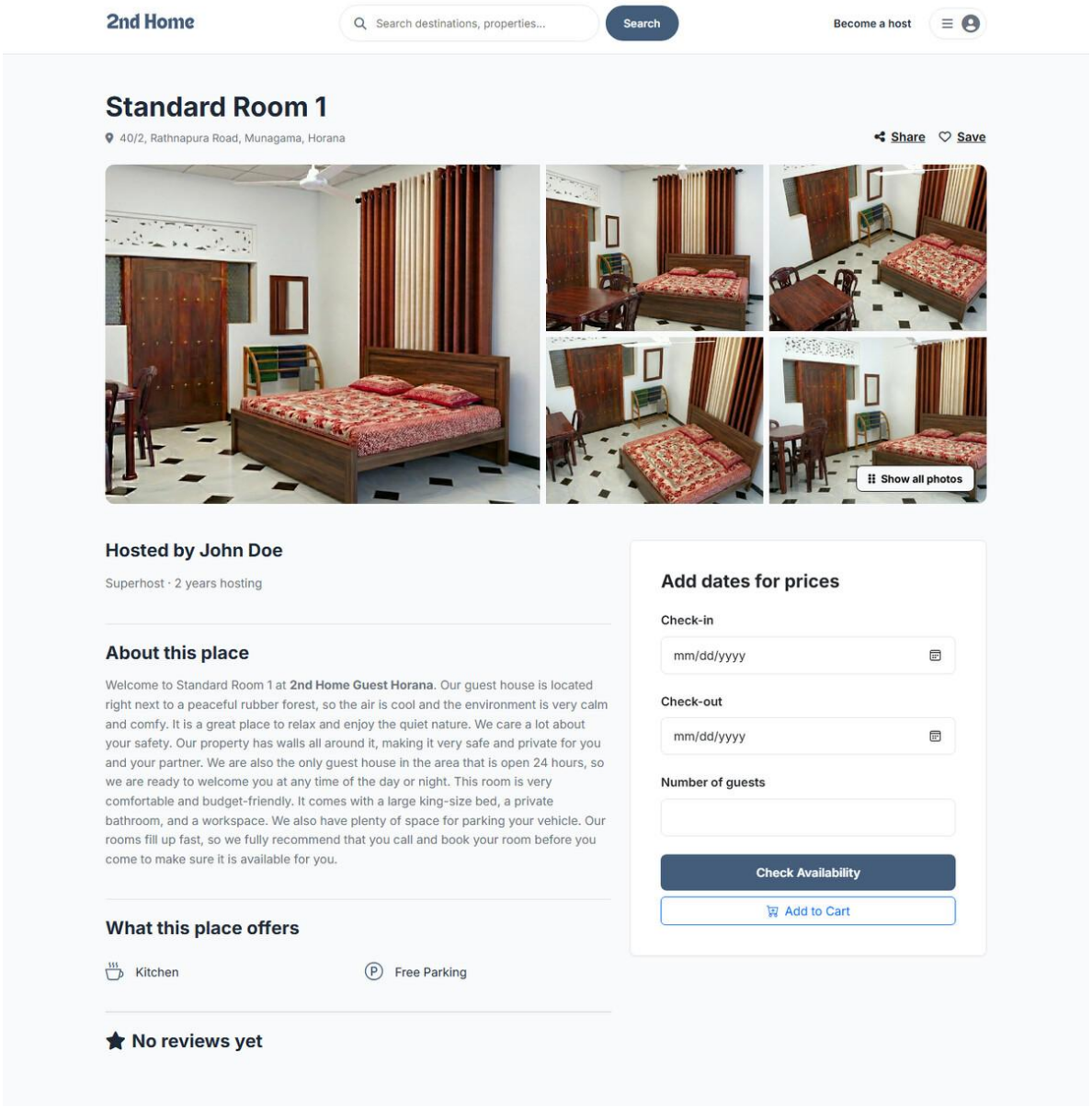


Figure 11.3 – Property page with gallery, amenities, reviews and booking card

11.4 Confirm and Pay Page

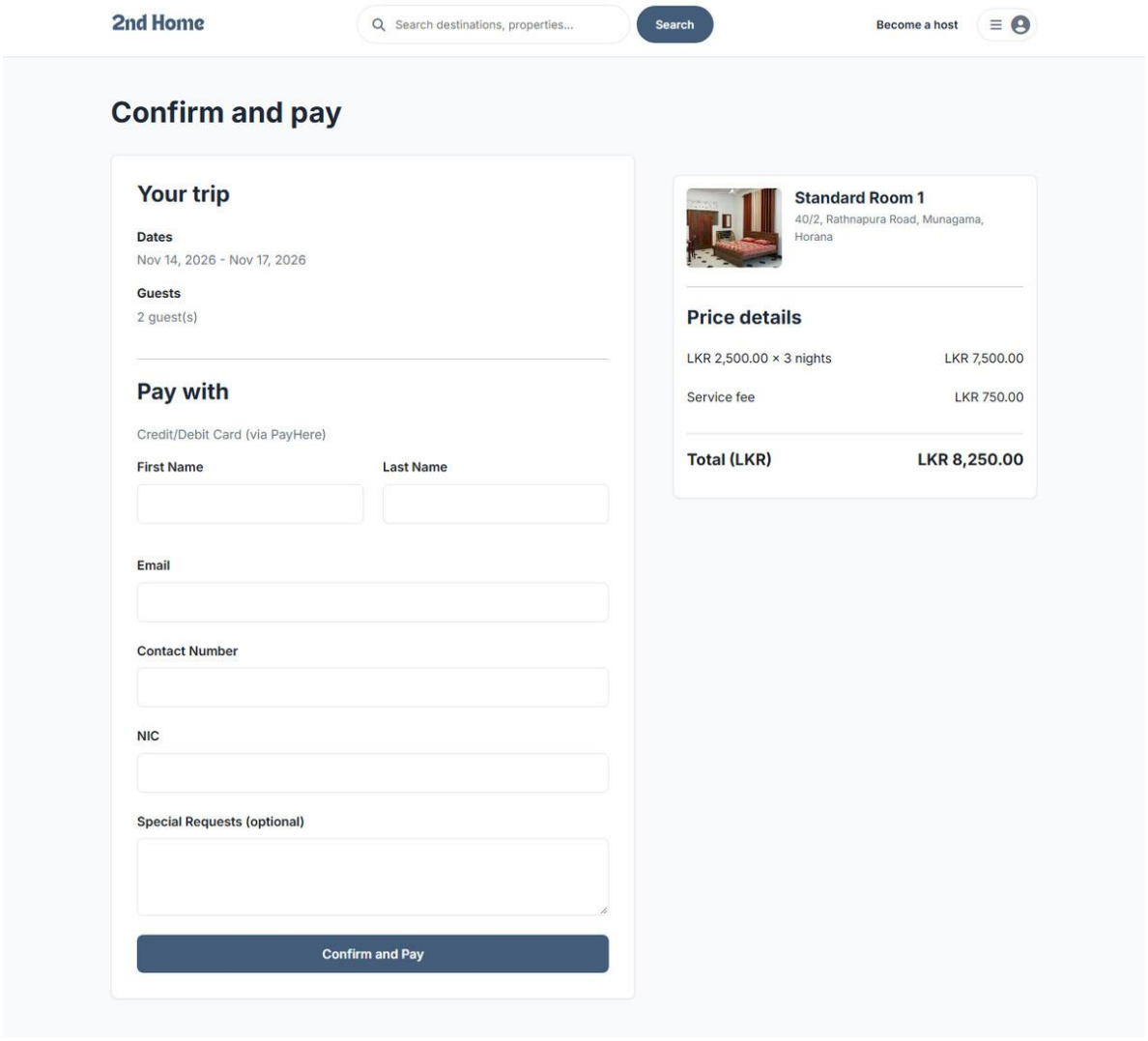


Figure 11.4 – Booking page with price breakdown (PayHere payment)

11.5 Sign Up Page

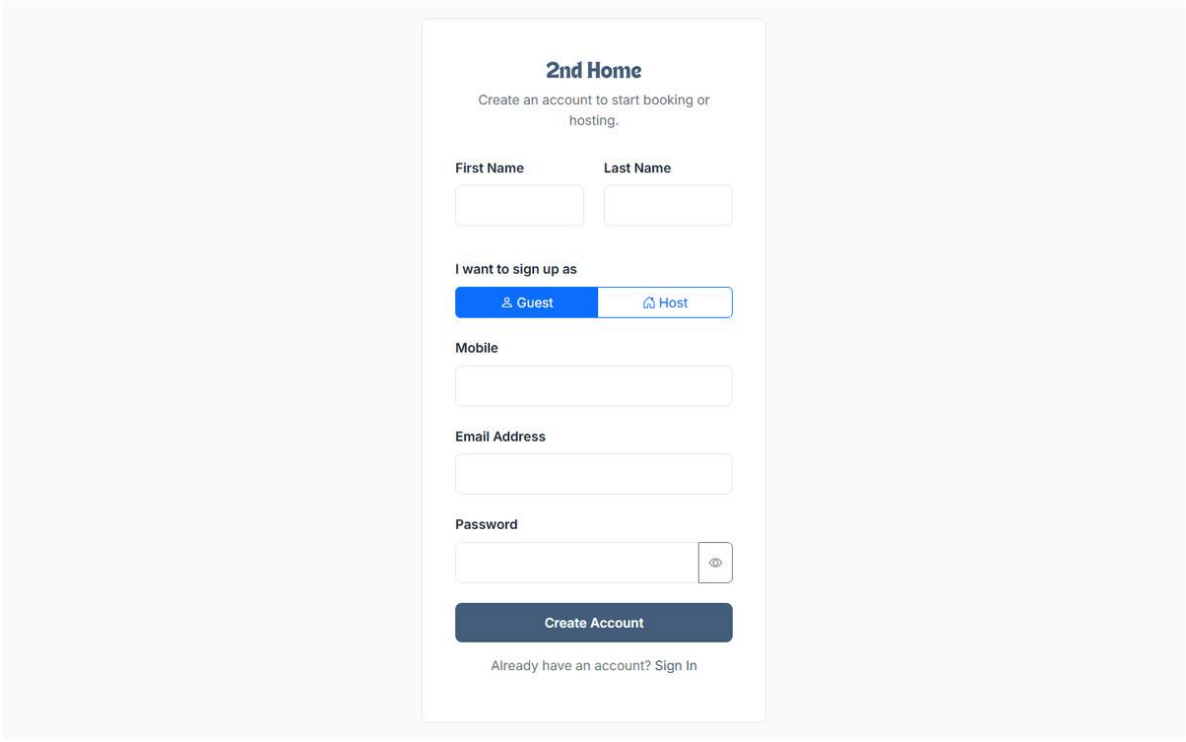


Figure 11.5 – Sign up page (guest or host)

11.6 Sign Up Validation Error

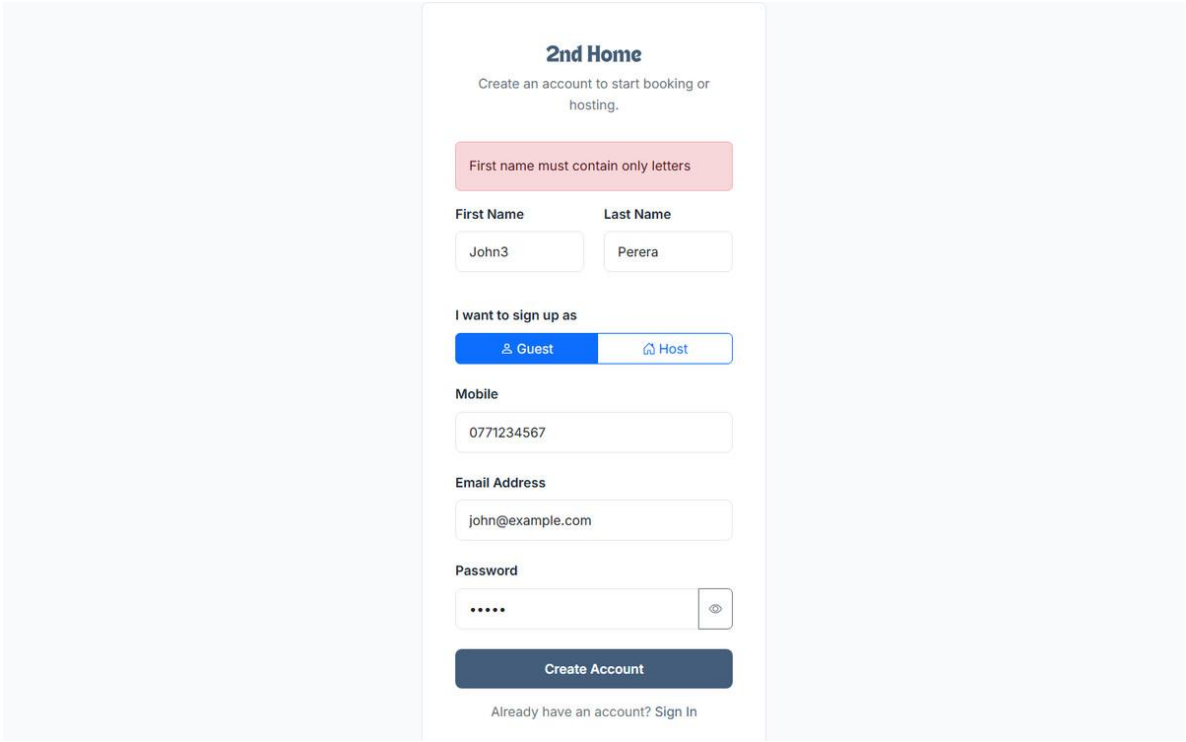


Figure 11.6 – Validation message on the sign up form

11.7 Sign In Page

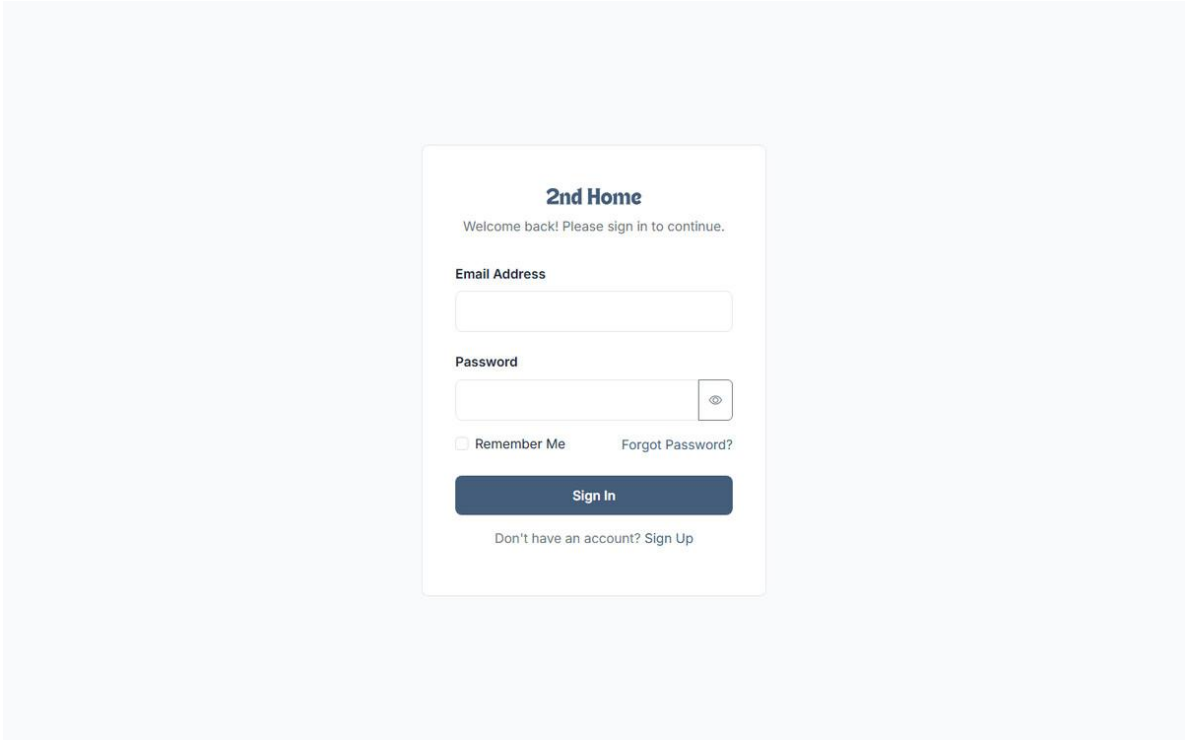


Figure 11.7 – Sign in page

11.8 Sign In Error Message

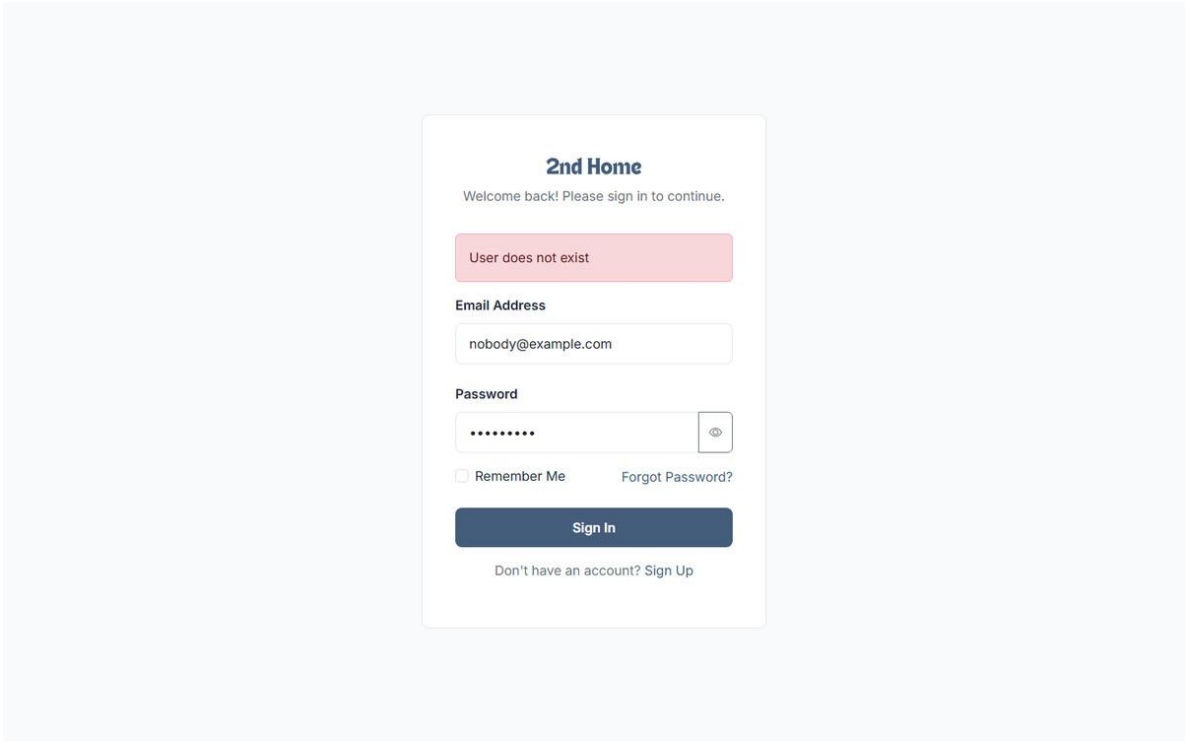


Figure 11.8 – Error shown for wrong login details

11.9 Forgot Password Page

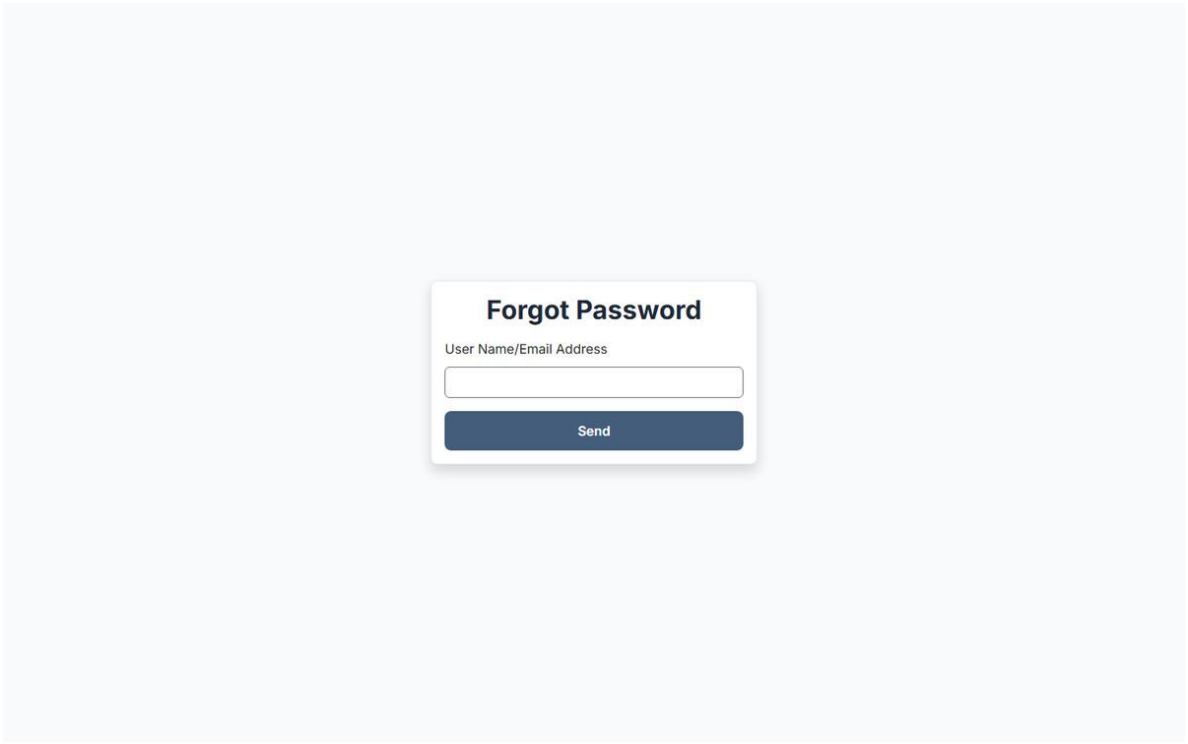


Figure 11.9 – Forgot password page

11.10 My Cart

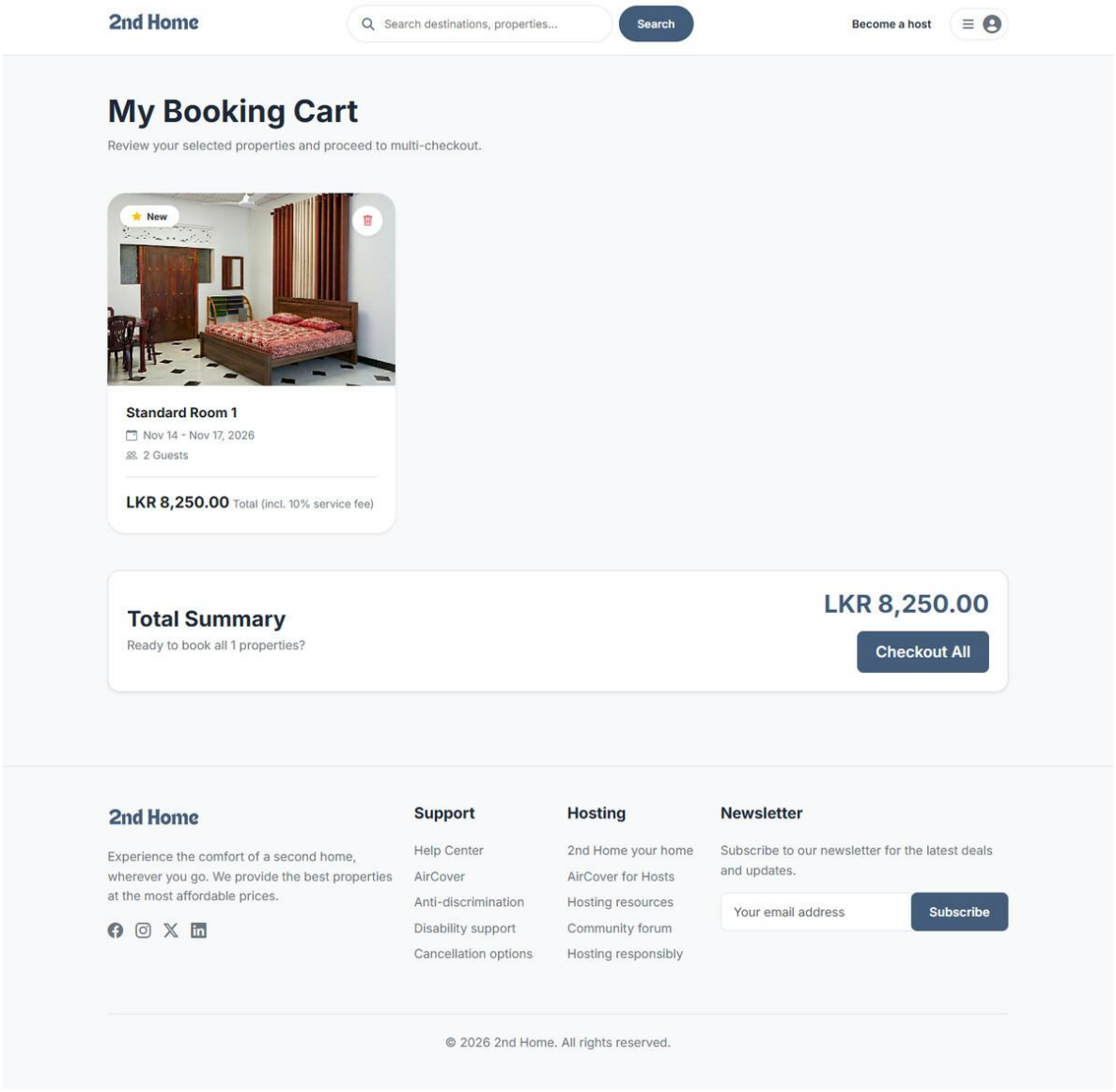


Figure 11.10 – Booking cart with multi-checkout

11.11 My Favorites

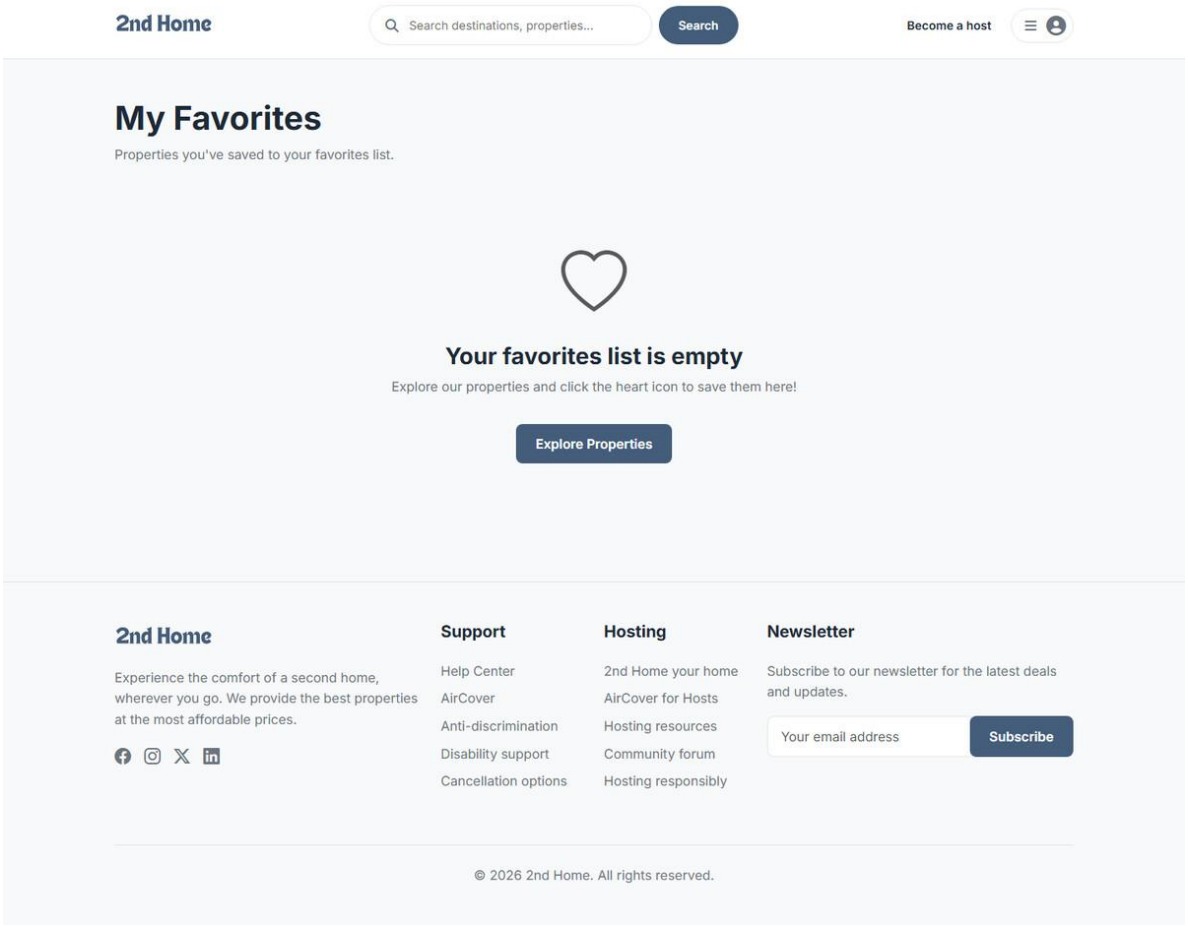


Figure 11.11 – Favorites page

11.12 My Bookings

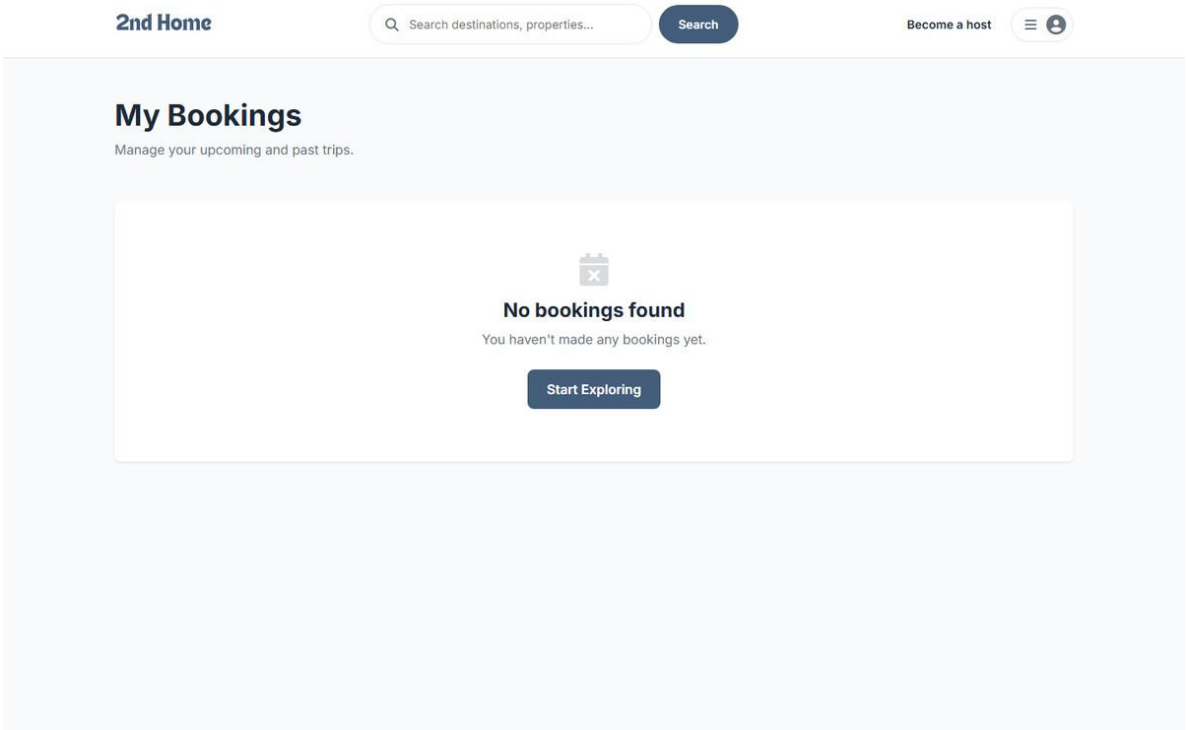


Figure 11.12 – My Bookings page (no bookings yet)

11.13 Account Settings

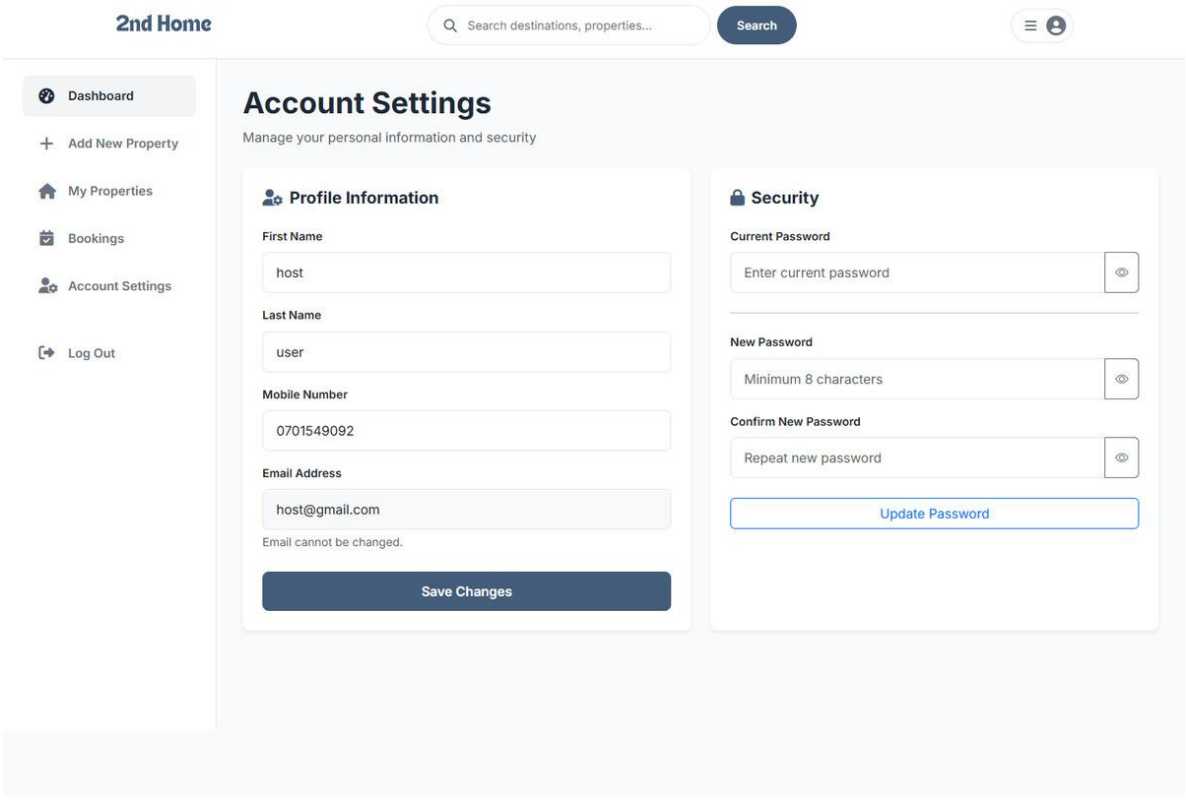


Figure 11.13 – Account settings (profile and password)

11.14 Host Dashboard

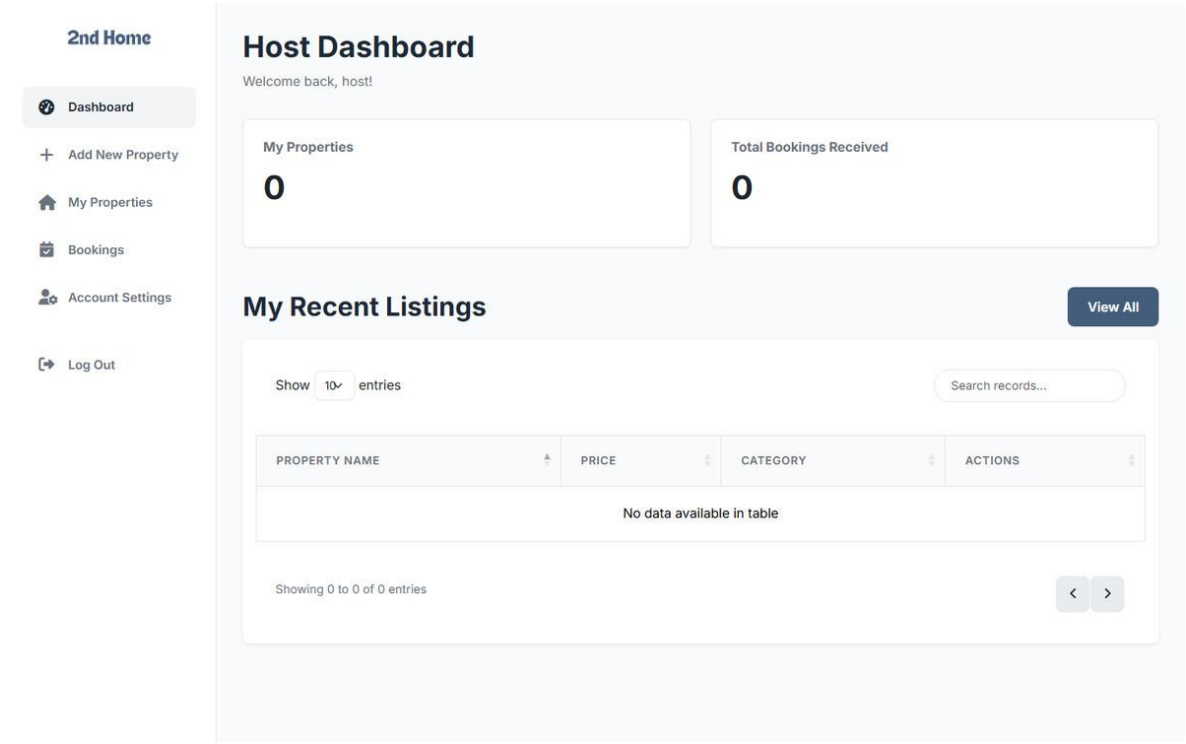


Figure 11.14 – Host dashboard

11.15 Host – My Properties

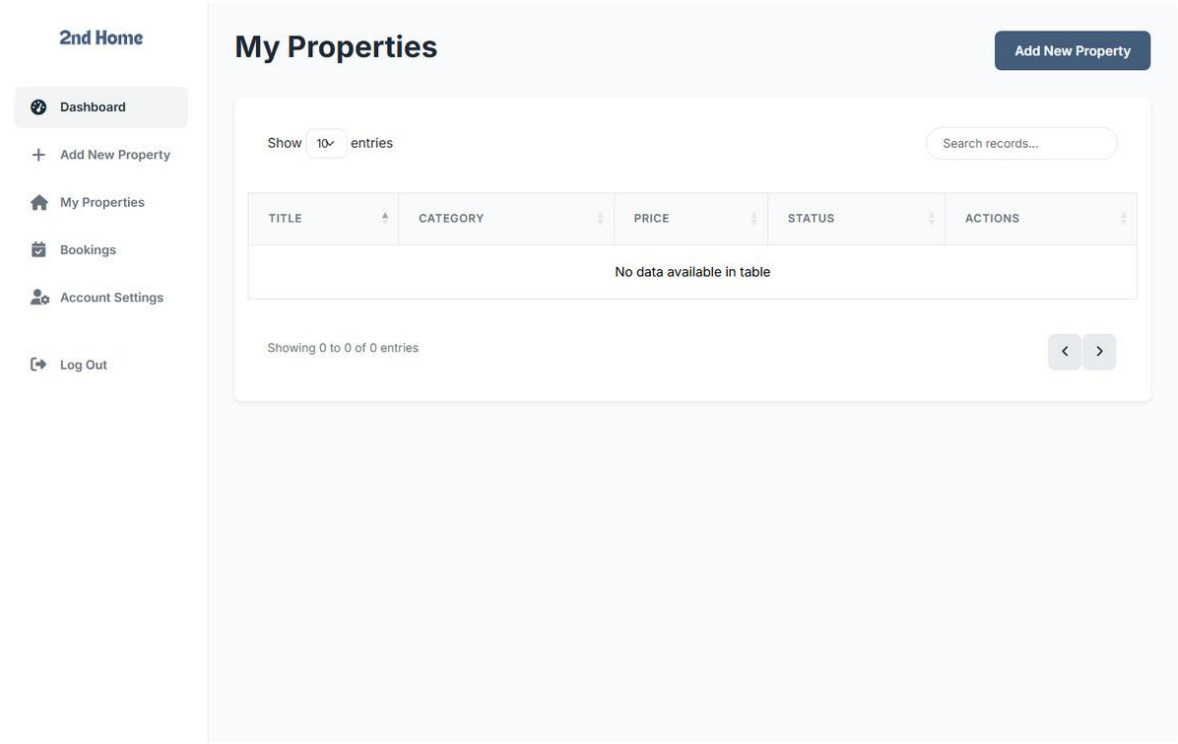


Figure 11.15 – Host property list

11.16 Host – Add New Property

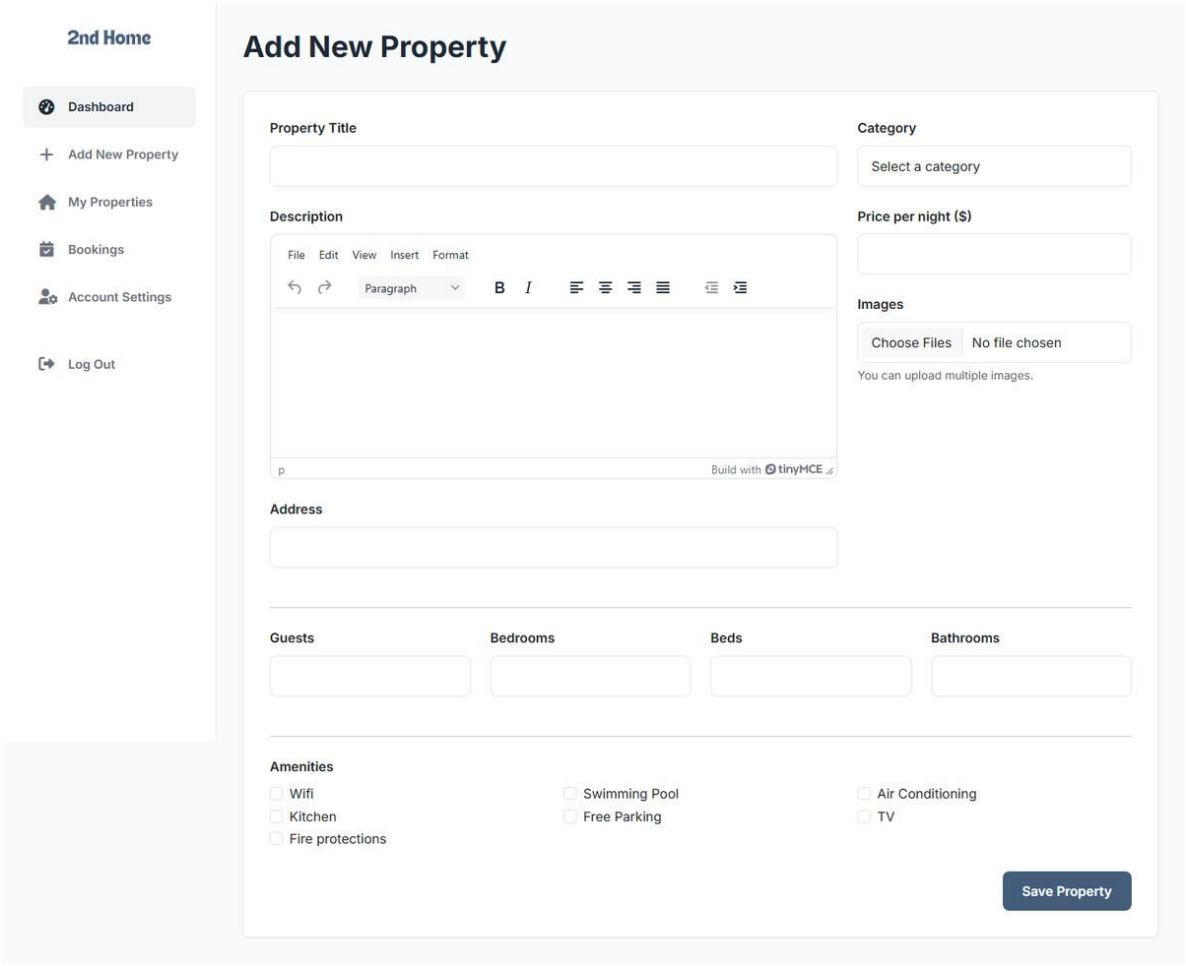


Figure 11.16 – Add property form with rich-text editor

11.17 Host – Bookings

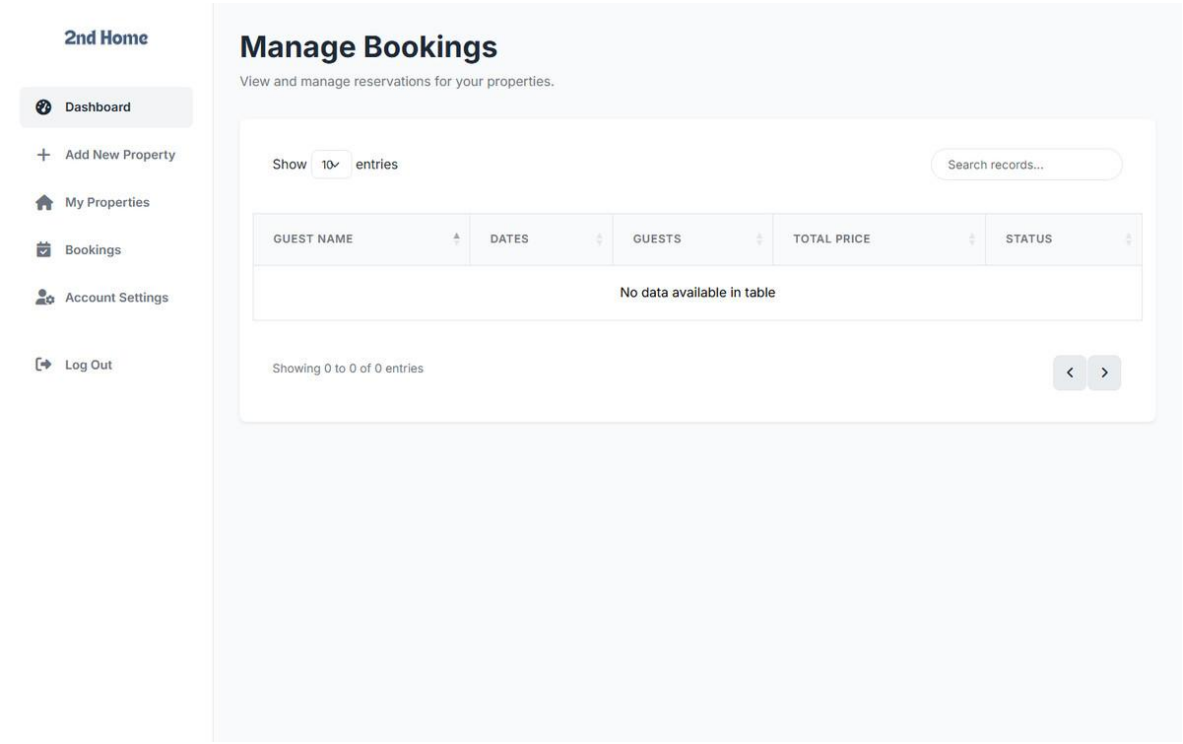


Figure 11.17 – Bookings received by the host

11.18 Admin Dashboard

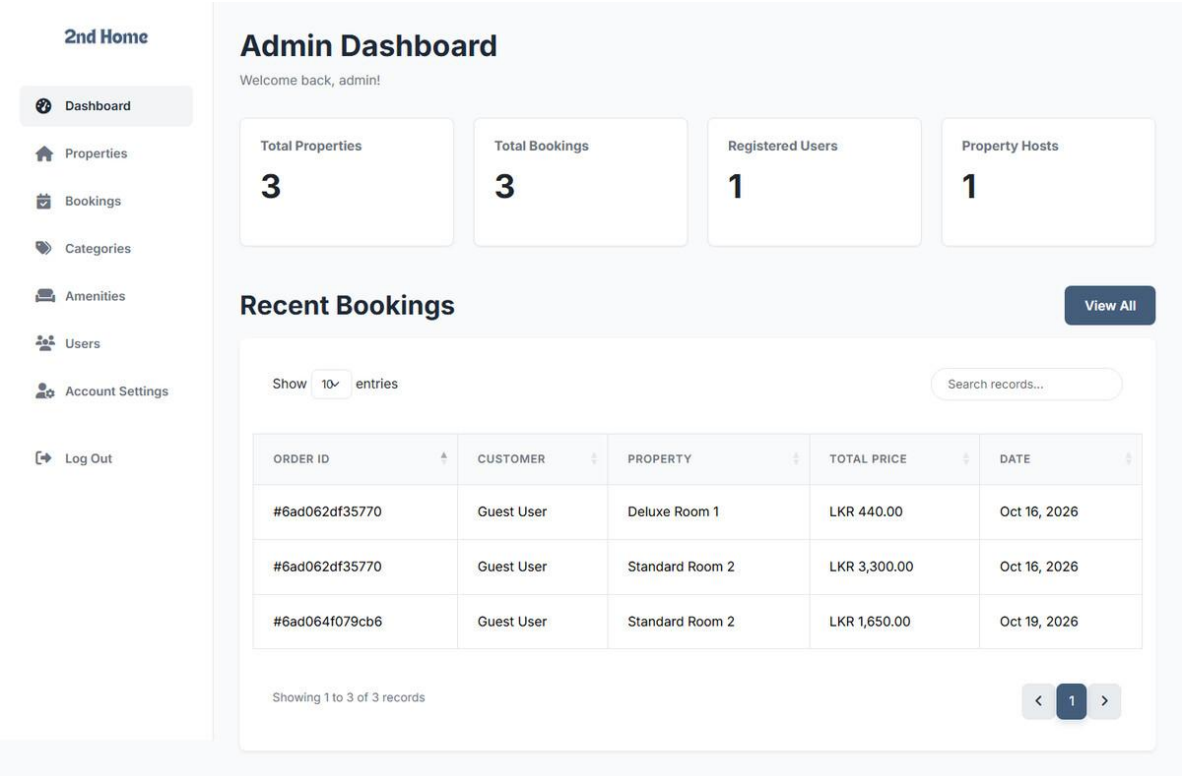


Figure 11.18 – Admin dashboard with summary statistics

11.19 Admin – Manage Properties

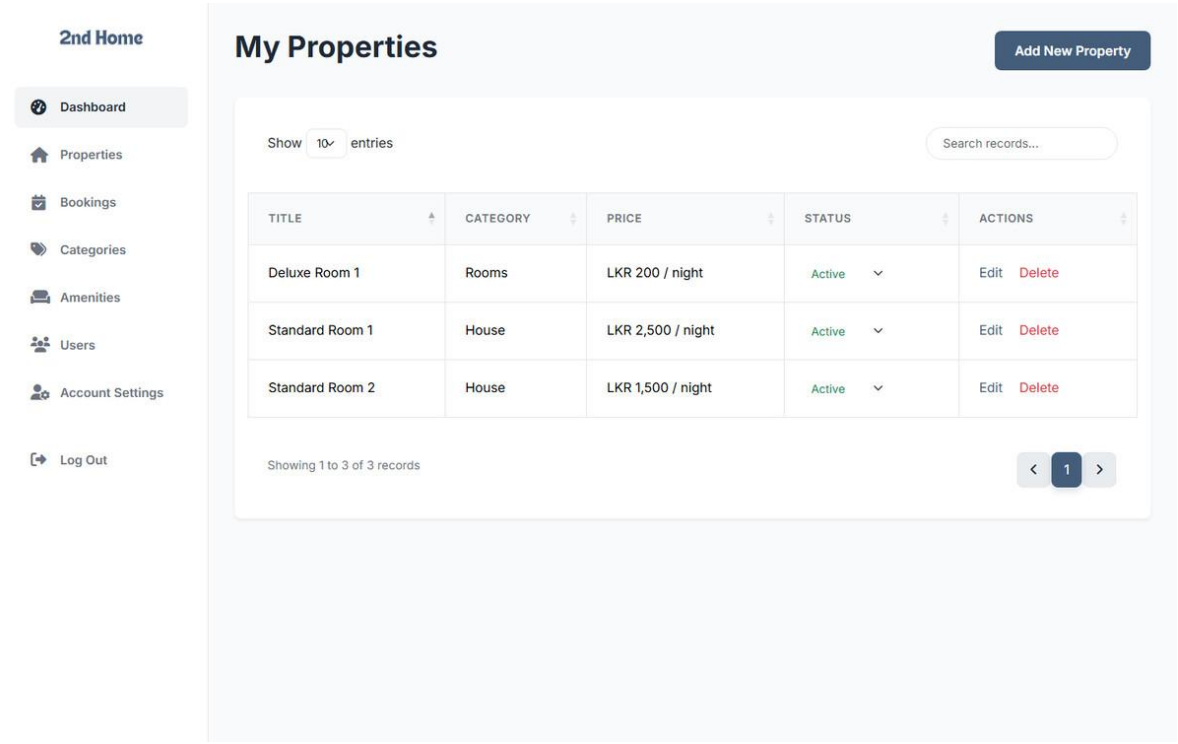


Figure 11.19 – Admin property list

11.20 Admin – Edit Property

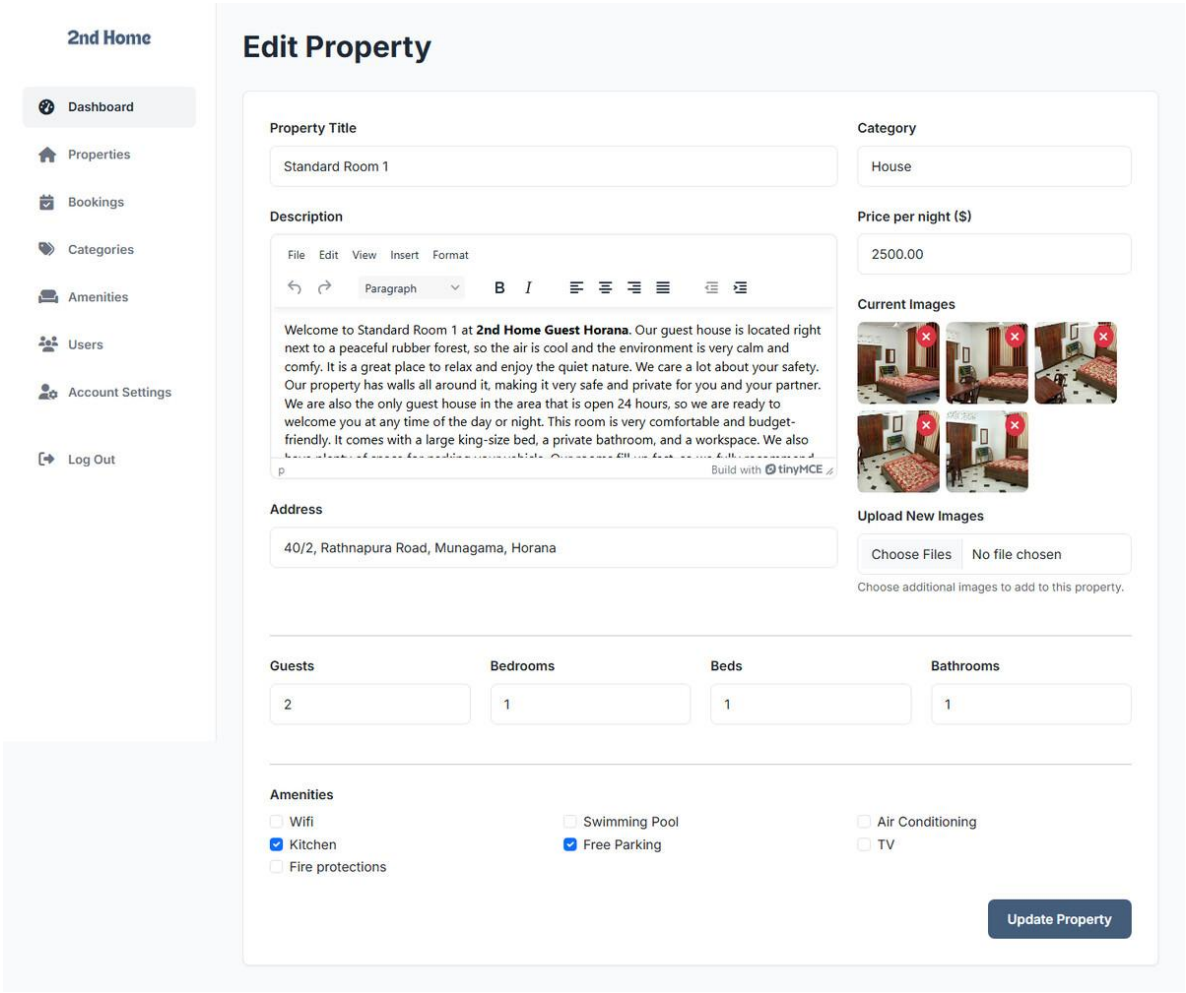


Figure 11.20 – Edit property form

11.21 Admin – Manage Bookings

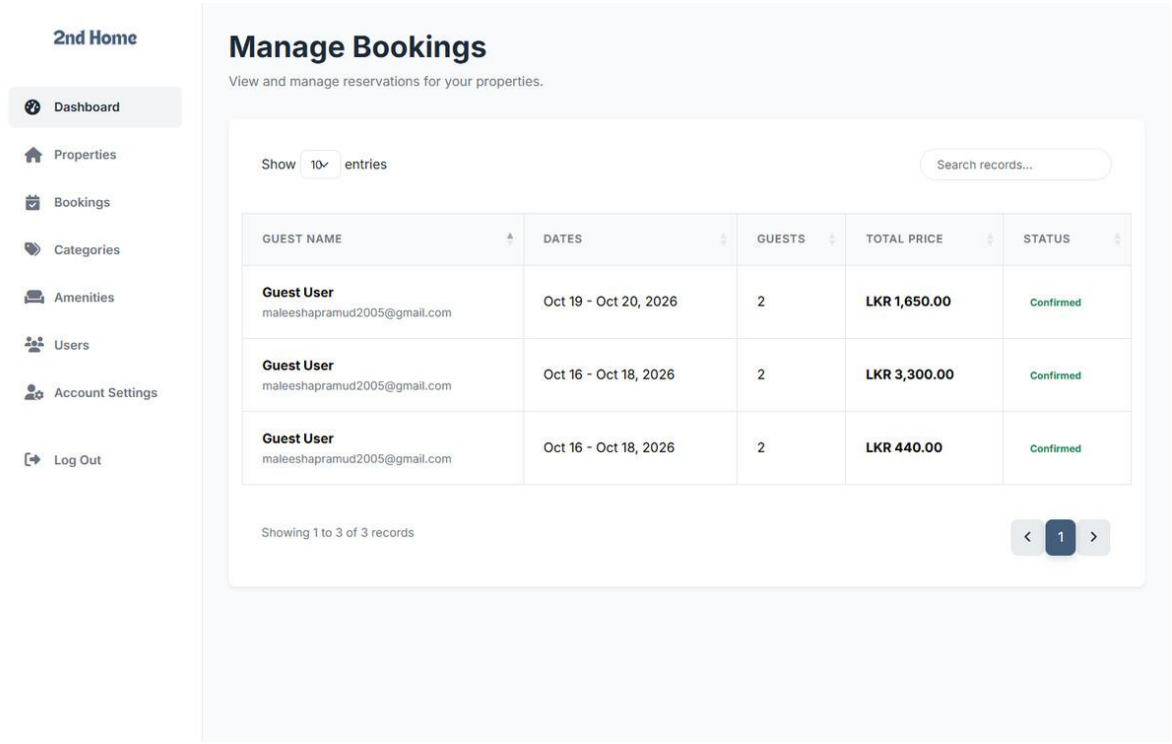


Figure 11.21 – All bookings

11.22 Admin – Categories

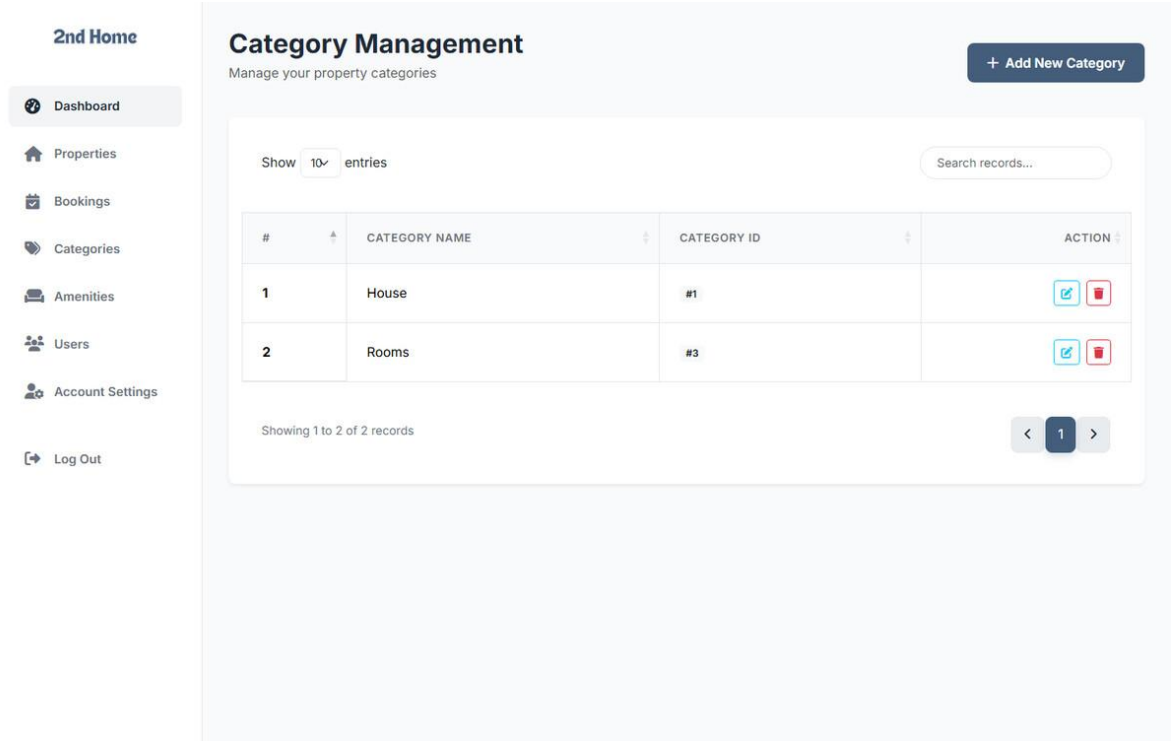


Figure 11.22 – Category list

11.23 Admin – Add Category

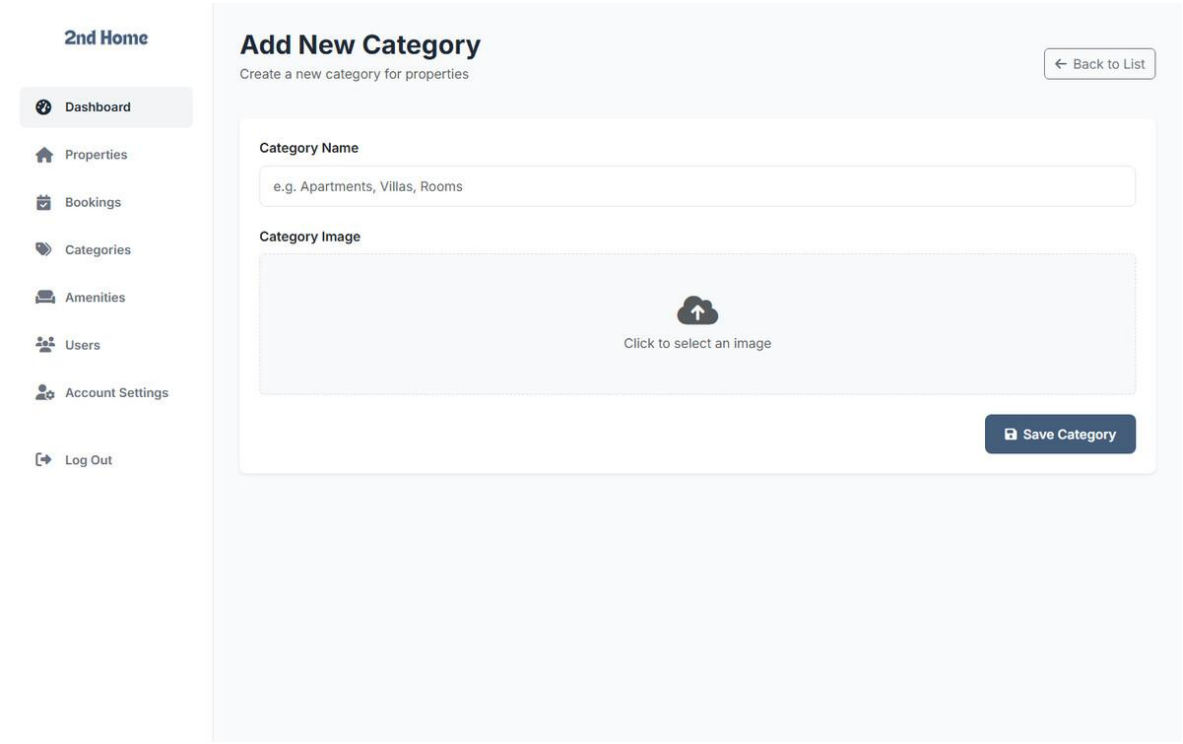


Figure 11.23 – Add category form

11.24 Admin – Amenities

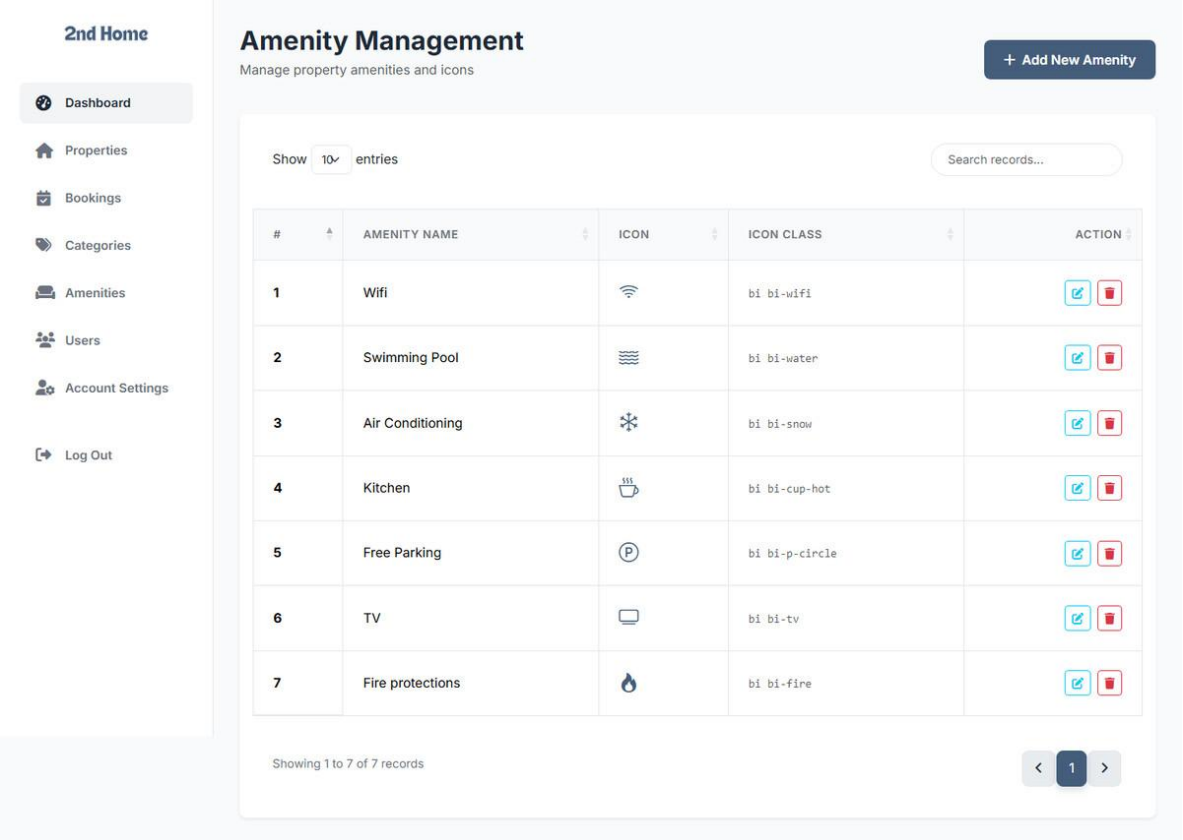


Figure 11.24 – Amenity list

11.25 Admin – User Management

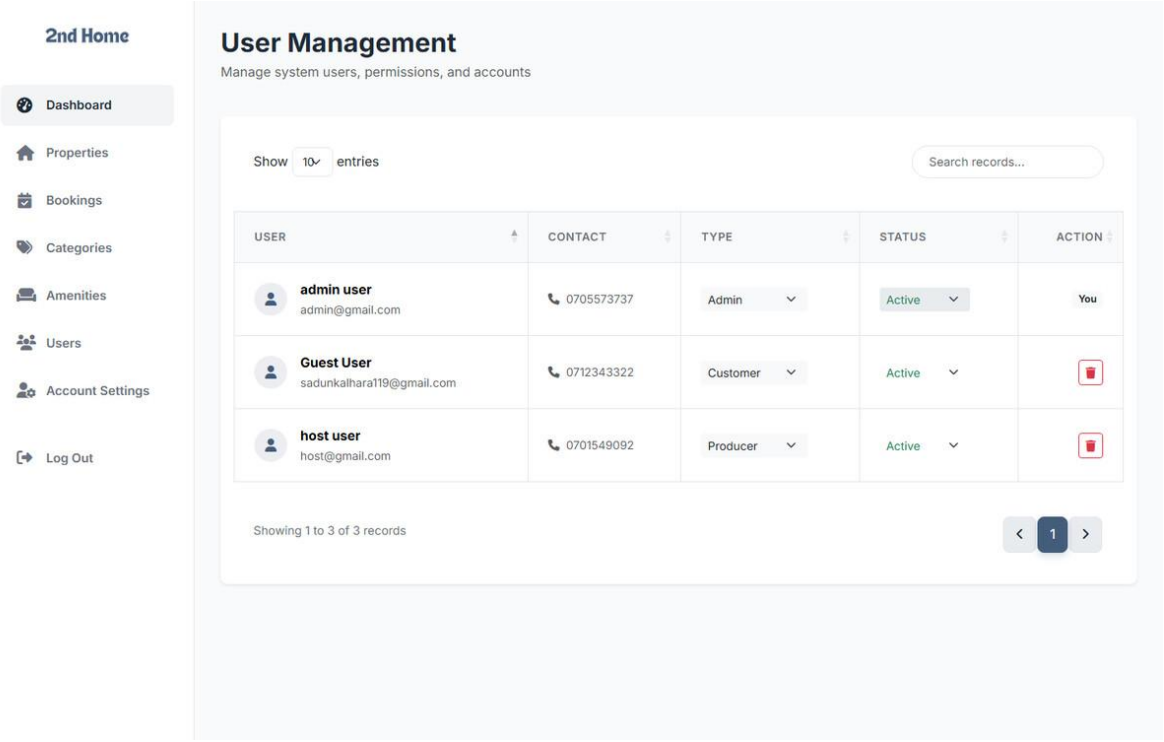


Figure 11.25 – User management (type and status)

11.26 XML Feed

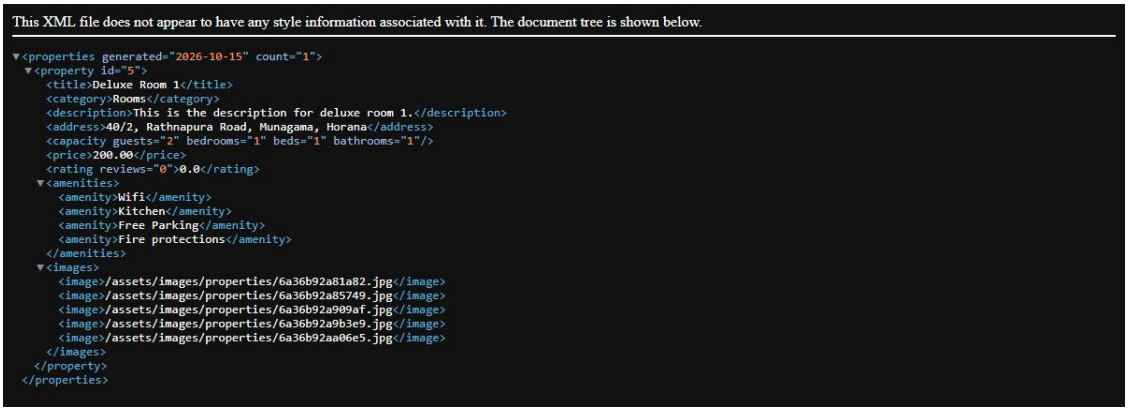


Figure 11.26 – XML feed of the properties

11.27 XML Validation Result



Figure 11.27 – XML checked against the DTD

11.28 XML Feed Link in the Footer

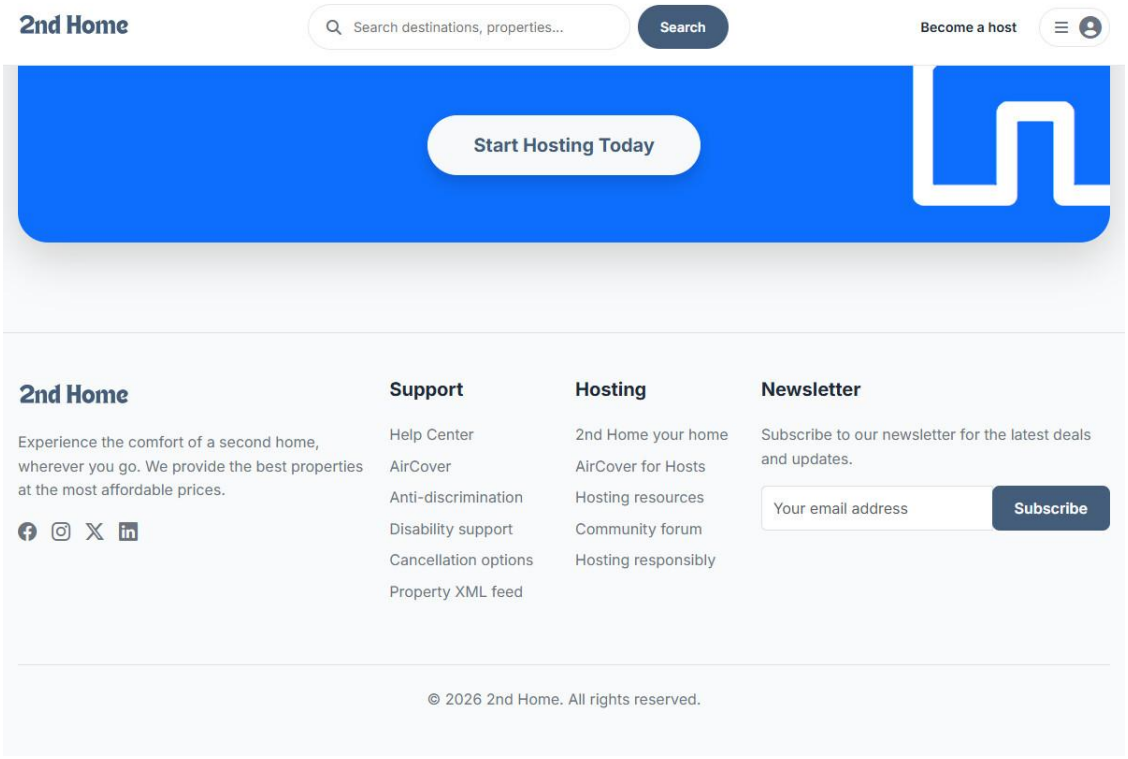


Figure 11.28 – Property XML feed link in the website footer

12. Conclusion

The 2nd Home web application has been designed and built successfully. It replaces phone calls and handwritten calendars with a fast, easy-to-use website that guests, hosts and administrators can use from any device.

What the system does

- Guests can search properties, check availability, book online and pay through PayHere with fully validated forms.
- Guests can use a booking cart for multi-checkout, save favourites and review the properties they have stayed in.
- Hosts can add, edit, activate and delete their own listings with photos, amenities and rich-text descriptions, and see their bookings.
- Administrators can manage properties, categories, amenities, users and bookings from one dashboard.
- Double bookings are prevented, prices are calculated on the server and every form is validated on both the browser and the server.
- Confirmation, cancellation, welcome and password-reset e-mails are sent automatically.
- An XML feed of the properties is published and checked against a DTD (properties-feed.php and xml/properties.dtd).
- The code is organised with reusable components, a Database class, a Validation class and a Mailer class.
- The interface is responsive and uses a consistent theme defined by CSS variables.

Future improvements

The system can be improved in the future by:

- Storing passwords as hashes with password_hash() / password_verify() instead of plain text, and removing the password from cookies and the session.
- Replacing escaped SQL strings with prepared statements (bind_param) for every query.
- Checking the user role inside every admin and host process file, and adding CSRF tokens to forms.
- Moving the database, SMTP and PayHere secrets out of the source code into environment variables.
- Completing PayHere payment verification with a notify URL, and building a mobile app that reads the XML feed.
- Adding host replies to reviews, a booking calendar view and online refunds.

Appendices

Project File Structure

```
onlinestore/
  index.php                Home page
  search.php               Search and category results
  single-property.php      Property details, availability, reviews
  booking.php              Confirm and pay
  cart.php                 Booking cart (multi-checkout)
  my-bookings.php          Guest bookings (edit / cancel)
  favorites.php            Saved properties
  account-settings.php     Profile and password
  signin.php  signup.php  logout.php
  forgot-password.php  reset-password.php
  properties-feed.php      XML feed of the properties
  database.sql             Database script
  Dockerfile  docker-compose.yml
  admin/
    dashboard.php          Admin overview
    user-management.php    Users: type, status, delete
    category/              list.php, add.php, edit.php
    amenity/               list.php, add.php, edit.php
  producer/
    dashboard.php          Host overview
  listing/
    list.php  add.php  edit.php  bookings.php
  components/
    head.php  NavigationBar.php  Footer.php  script.php
    AdminSidebar.php  ProducerSidebar.php  FilterForm.php  email.php
  includes/
    connection.php         Database class
    validation.php         Validation class
    mailer.php             Mailer class (PHPMailer)
  lib/                     AJAX process files (*-process.php)
  mail/                    PHPMailer library
  xml/
    properties.dtd         DTD rules for the XML feed
  migrations/              SQL updates (cart, status, reviews)
  assets/
    css/  js/  libraries/
    images/  banner  categories  logo  properties  slider
```

Database Schema

Table	Column	Type & Notes
user_types	id / name	INT PK AUTO / VARCHAR(50)
status	id / name	INT PK AUTO / VARCHAR(50)
users	id	INT, PK, AUTO – Primary key
users	fname, lname	VARCHAR(50) each
users	mobile	VARCHAR(20)
users	email	VARCHAR(100)
users	password	VARCHAR(255)
users	vcode	VARCHAR(255), NULL – password reset code

Table	Column	Type & Notes
users	user_type_id / status_id	INT – links to user_types / status (default status 1)
categories	id, name, image	INT PK AUTO, VARCHAR(100), VARCHAR(255) image filename
amenities	id, name, icon_cls	INT PK AUTO, VARCHAR(100), VARCHAR(100) icon class
properties	id	INT, PK, AUTO – Primary key
properties	title	VARCHAR(255)
properties	categories_id	INT – links to categories.id
properties	description, address	TEXT
properties	guests, bedrooms, beds, bathrooms	INT
properties	images	TEXT – comma-separated photo file names
properties	base_price	DECIMAL(10,2) – price per night
properties	users_id / status_id	INT – host (users.id) / Active or Inactive
properties_has_amenities	properties_id, amenities_id	INT – composite primary key
bookings	id	INT, PK, AUTO – Primary key
bookings	checkIn, checkOut	DATE
bookings	first_name, last_name	VARCHAR(50)
bookings	nic, contact	VARCHAR(20)
bookings	guests, properties_id	INT – properties_id links to properties.id
bookings	email	VARCHAR(100)
bookings	special_requests	TEXT, NULL
bookings	total_price	DECIMAL(10,2)
bookings	order_id	VARCHAR(100) – PayHere order id
cart	id	INT, PK, AUTO – Primary key
cart	users_id, properties_id	INT, FK – ON DELETE CASCADE
cart	checkIn, checkOut, guests	DATE, DATE, INT
reviews	id	INT, PK, AUTO – Primary key
reviews	properties_id, users_id	INT, FK – ON DELETE CASCADE
reviews	rating	INT (1–5)
reviews	comment	TEXT
reviews	created_at	TIMESTAMP – default current time

Demo Log In Credentials

Accounts that exist in the development database:

Role	Email	Password
Administrator	admin@gmail.com	Password123
Host (Producer)	host@gmail.com	Password123